Guia 5

Formación estelar / Datos SDSS y comparación con un semi-analítico

Paquetes

In [1]:
```
from astropy.cosmology import Planck18 as cosmo
from IPython.display import display, Math
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import ks_2samp
import scienceplots
import seaborn as sns
import sys
sys.path.append('../')
from guia2.bimodal import fit_bimodal, normal, bimodal
```

In [2]:
```
plt.style.use('science')
_params = {
    'figure.figsize':(5*1.5,4*1.5),
    #'figure.figsize':(6.47,4), # golden ratio
    'figure.dpi':96,
    'font.family':'sans-serif',
    'font.size':15,
    'savefig.format':'png',
    'savefig.transparent':True,
    'xtick.direction':'in',
    'xtick.top':True,
    'ytick.direction':'in',
    'ytick.right':True,
    'errorbar.capsize':3,
    'legend.loc':'upper right',
    'legend.frameon':True,
    'legend.fontsize':10,
    #'axes.axisbelow':True,
}
plt.rcParams.update(_params)
```

Constantes

In [3]:
```
CL = 'Cluster'
FD = 'Field'
SA = 'Semianalytic'

FILENAMES = {
    CL:'sample1_cls.dat',
    FD:'sample2_fd.dat',
    SA:'sample3_sim.dat'
}

COLUMNS = {
```

```
    CL:['cls', 'ra', 'dec', 'z',
        'Mr01', 'ur', 'mur', 'kr50',
        'C', 'stellarmass', 'sfr', 'ssfr',
        'Dn4000', 'class', 'Zc', 'sigma',
        'Mvir', 'Rvir', 'R200', 'w',
        'dist', 'Pe', 'Ps'
    ],
    FD:['ra', 'dec', 'z', 'Mr01',
        'ur', 'mur', 'kr50', 'C',
        'stellarmass', 'sfr', 'ssfr', 'Dn4000',
        'OH', 'w', 'Pe', 'Ps'
    ],
    SA:['cls', 'ig', 'type',  'clase',
        'stellarmass', 'u', 'g', 'r',
        'sfr', 'Mcoldgas', 'Mhotgas', 'Tau',
        'OH', 'bt', 'ssfr'
    ]
}

COLORS = {
    CL:'#7b3294',
    FD:'#008837',
    SA:'#ff7f00'
}
```

In [51]:
```
GRID = 50
LEVELS = 5
LWS = np.linspace(0.8,2.5,LEVELS)
```

Funciones

In [71]:
```
def sSFR_threshold(z):
    return np.log10(0.2/cosmo.age(z).to('yr').value)
```

In [5]:
```
def fitbimodal_quantiles(data, col_fit='ur', col_q='Mr01', nquantiles=5,
    G = data[col_q]
    C_fit = data[col_fit]
    mask = []
    quantiles = []
    mean_x = []

    q = np.quantile(G, np.linspace(0.0,1.0,nquantiles+1)[1:-1])

    #print('data<q[0]')
    mask.append(G<q[0])
    quantiles.append(f'${G}<{q[0]:2.2f}$')
    mean_x.append(0.5*(G.min()+q[0]))
    for i in range(nquantiles-1):
        if i<nquantiles-2:
            #print(f'data>q[{i}] and data<q[{i+1}]')
            mask.append((G>q[i])&(G<q[i+1]))
            quantiles.append(f'${q[i]:2.2f}<{col_q}<{q[i+1]:2.2f}$')
            mean_x.append(0.5*(q[i]+q[i+1]))
        else:
            #print(f'data>q[{i}]')
            mask.append(G>q[i])
            quantiles.append(f'${col_q}>{q[i]:2.2f}$')
            mean_x.append(0.5*(q[i]+G.max()))
```

```
    fits = []
    for m in mask:
        fits.append(
            fit_bimodal(
                C_fit[m],
                p0=p0,
                nbins=50
            )
        )
    return mean_x, fits
```

In [328…
```
def weighted_median(df, val, weight='w'):
    df_sorted = df.sort_values(val)
    cumsum = df_sorted[weight].cumsum()
    cutoff = df_sorted[weight].sum() / 2.
    return df_sorted[cumsum >= cutoff][val].iloc[0]
```

In [555…
```
def median_of_col(data, nbins=10, col_median='dist', col_corr='Dn4000'):
    bin_edges = np.linspace(data[col_median].min(), data[col_median].max(
    bin_center = 0.5*(bin_edges[1:]+bin_edges[:-1])
    median = np.zeros(nbins)
    std = np.zeros(nbins)
    for i in range(nbins):
        mask = (data[col_median]>bin_edges[i])&(data[col_median]<bin_edge
        median[i] = np.median(data[col_corr][mask])
        std[i] = np.std(data[col_corr][mask])

    return bin_center, median, std
```

In [525…
```
def weighted_mean_of_col(data, nbins=10, col_median='dist', col_corr='Dn4
    bin_edges = np.linspace(data[col_median].min(), data[col_median].max(
    bin_center = 0.5*(bin_edges[1:]+bin_edges[:-1])
    median = np.zeros(nbins)
    std = np.zeros(nbins)
    for i in range(nbins):
        mask = (data[col_median]>bin_edges[i])&(data[col_median]<bin_edge
        if not np.any(mask): continue
        median[i] = np.average(data[col_corr][mask], weights=data['w'][ma
        std[i] = np.std(data[col_corr][mask])

    return bin_center, median, std
```

Catálogos

`sample1.dat` : Galaxias en cúmulos brillantes en rayos X

columnas	descripción
'z'	Redshift de la galaxia
'Mr01'	Magnitud absoluta en la banda r a 0.1
'ur'	Color u-r
'mur'	Brillo superficial
'kr50'	Radio que encierra el %50 del brillo $[\mathrm{kpc}]$

columnas	descripción
'C'	Índice de concentración
'stellarmass'	Masa estelar $[\log \mathrm{M}_\odot]$
'sfr'	Tasa de formación estelar $[M_\odot \mathrm{yr}^{-1}]$
'ssfr'	Tasa de formación estelar especifica $[\mathrm{yr}^{-1}]$
'Dn4000'	Break de los 4000Å $[\mathrm{ergs}^{-1}\ \mathrm{cm}^{-2}\ \mathrm{Hz}^{-1}]$
'class'	Tipo de galaxia en el cúmulo (-1,1,2,3,4,5) (?)
'Zc'	Redshift del cúmulo
'sigma'	Dispersión de velocidades
'Mvir'	Masa encerrada al radio virial
'Rvir'	Radio virial
'R200'	Radio que encierra 200 veces la densidad media del Universo
'w'	Peso $1/V_{max}$
'dist'	Distancia al centro del cumulo normalizada por R200
'Pe'	Probabilidad de ser eliptica, según galaxy-zoo
'Ps'	Probabilidad de ser espiral, según galaxy-zoo

`sample2.dat` : Galaxias de campo

columnas	descripción
'z'	Redshift
'Mr01'	Magnitud absoluta en la banda r a 0.1
'ur'	Color u-r
'mur'	Brillo superficial
'kr50'	r_50 $[\mathrm{kpc}]$
'C'	Índice de concentración
'stellarmass'	Masa estelar $[\log \mathrm{M}_\odot]$
'sfr'	Tasa de formación estelar [$M_\odot \mathrm{yr}^{-1}$]
'ssfr'	Tasa de formación estelar especifica $[\mathrm{yr}^{-1}]$
'Dn4000'	Break de los 4000Å $[\mathrm{ergs}^{-1}\ \mathrm{cm}^{-2}\ \mathrm{Hz}^{-1}]$
'OH'	Metalicidad $12 + \log[O/H]$
'w'	Peso $1/V_{max}$
'dist'	Distancia al centro del cúmulo normalizada al R_{200}
'Pe'	Probabilidad de ser elíptica, según galaxy-zoo
'Ps'	Probabilidad de ser espiral, según galaxy-zoo

`sample3.dat` : Modelo semianalítico

columnas	descripción
'cls'	Cluster id
'ig'	Galaxy id
'type'	(?)
'stellarmass'	Masa estelar
'u'	Magnitud en la banda u
'g'	Magnitud en la banda g
'r'	Magnitud en la banda r
'sfr'	Tasa de formación estelar $[M_\odot \mathrm{yr}^{-1}]$
'ssfr'	Tasa de formación estelar especifica $[\mathrm{yr}^{-1}]]$
'Mcoldgas'	Masa de gas frio [$\log M_\odot$]
'Mhotdgas'	Masa de gas caliente [$\log M_\odot$]
'Tau'	Edad de la galaxia [?]
'OH'	Metalicidad $12 + \log[O/H]$
'bt'	Tasa de masa bulge-to-total

In [6]:
```
samples = {}
for n in [CL, FD, SA]:
    samples[n] = pd.read_fwf(FILENAMES[n], names=COLUMNS[n], skiprows=1)
```

In [7]:
```
# Agregar color u-r al semianalítico
COLUMNS[SA].append('ur')
COLUMNS[SA].append('gr')
samples[SA]['ur'] = samples[SA]['u']-samples[SA]['r']
samples[SA]['gr'] = samples[SA]['g']-samples[SA]['r']
```

In [8]:
```
# Quitando outliers
samples[CL].query('Rvir < 30 and C<4.5 and Dn4000<2.5 and Dn4000>0.5', in
samples[FD].query('stellarmass > 8 and sfr > -8 and ur < 5 and kr50 < 20
samples[SA].query('Mhotgas != 99 and Mcoldgas != 99 and sfr!=99', inplace
```

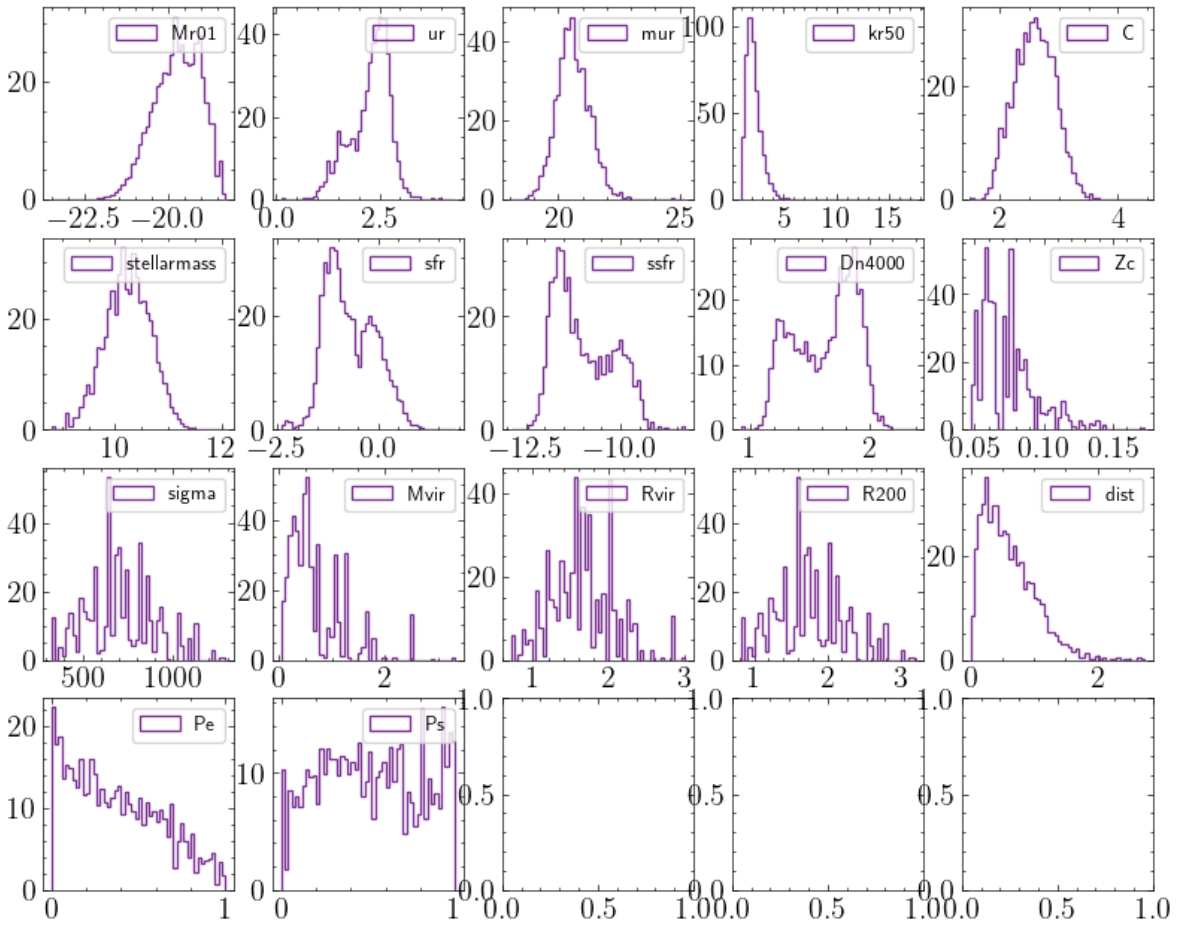
In [9]:
```
# Para automatizar el codigo, agrego una columna de "pesos" al semianalit
COLUMNS[SA].append('w')
samples[SA]['w'] = np.ones(len(samples[SA]['cls']))
```

Propiedades de Clusters

In [10]:
```
cluster = samples[CL]
```

In [11]:
```
# Distribuciones de las columnas, pesadas por 'w'
fig, axes = plt.subplots(4,5,figsize=(10,8))
ax = axes.flatten()
i=0
for col in COLUMNS[CL]:
```

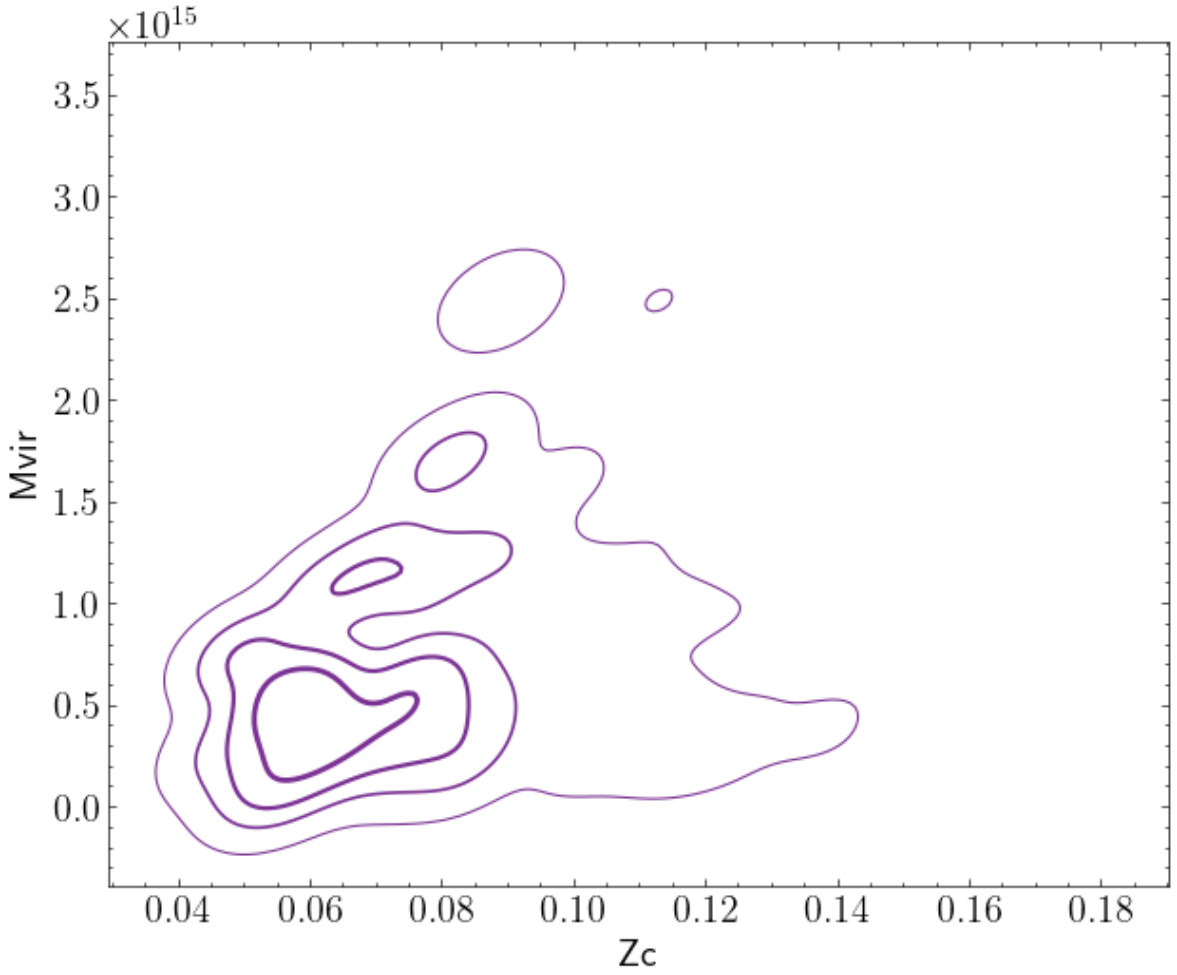
```
    if col=='cls': continue
    if col=='ra': continue
    if col=='dec': continue
    if col=='z': continue
    if col=='class': continue
    if col=='w': continue
    ax[i].hist(cluster[col], bins=50, weights=cluster['w'], histtype='ste
    ax[i].legend()
    i+=1
```



In [212…

```
fig, ax = plt.subplots()
sns.kdeplot(
    ax=ax,
    data=cluster,
    x='Zc', y='Mvir',
    weights='w',
    color=COLORS[CL], levels=LEVELS, linewidths=LWS
)
```

Out[212… <Axes: xlabel='Zc', ylabel='Mvir'>



In [402…
```
mask = np.random.randint(2,size=len(cluster),dtype=bool)
np.sum(cluster[mask]['w'])
```

Out[402… 241.845

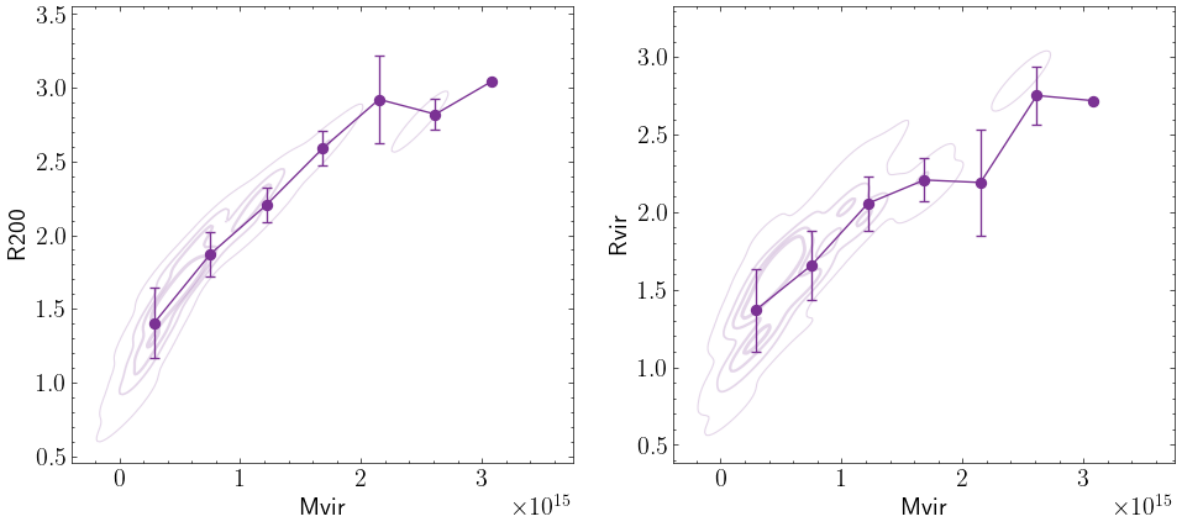
In [411…
```
std
```

Out[411…
```
array([0.26678632, 0.24409102, 0.22496578, 0.21684326, 0.21092693,
       0.20621285, 0.18985668, 0.10160463, 0.        , 0.        ])
```

In [412…
```
fig, axes = plt.subplots(1,2, figsize=(12,5))
sns.kdeplot(
    ax=axes[0],
    data=cluster,
    x='Mvir', y='R200',
    weights='w',
    color=COLORS[CL], levels=LEVELS, linewidths=LWS,
    alpha=0.2
)
bin_center, median, yerr = weighted_mean_of_col(cluster, nbins=7, col_med
axes[0].errorbar(
    bin_center, median, yerr=yerr,
    c=COLORS[CL],
    fmt='o-'
)

sns.kdeplot(
    ax=axes[1],
    data=cluster,
    x='Mvir', y='Rvir',
```

```
    weights='w',
    color=COLORS[CL], levels=LEVELS, linewidths=LWS,
    alpha=0.2
)
bin_center, median, yerr = weighted_mean_of_col(cluster, nbins=7, col_med
axes[1].errorbar(
    bin_center, median, yerr=yerr,
    c=COLORS[CL],
    fmt='o-'
)

#sns.kdeplot(
#    ax=axes[1,0],
#    data=cluster,
#    x='Rvir', y='R200',
#    weights='w',
#    color=COLORS[CL], levels=LEVELS, linewidths=LWS,
#    alpha=0.2
#)
#bin_center, median, yerr = median_of_col(cluster, col_median='Rvir', col
#axes[1,0].errorbar(
#    bin_center, median, yerr=std,
#    c=COLORS[CL],
#    fmt='o-'
#)
```

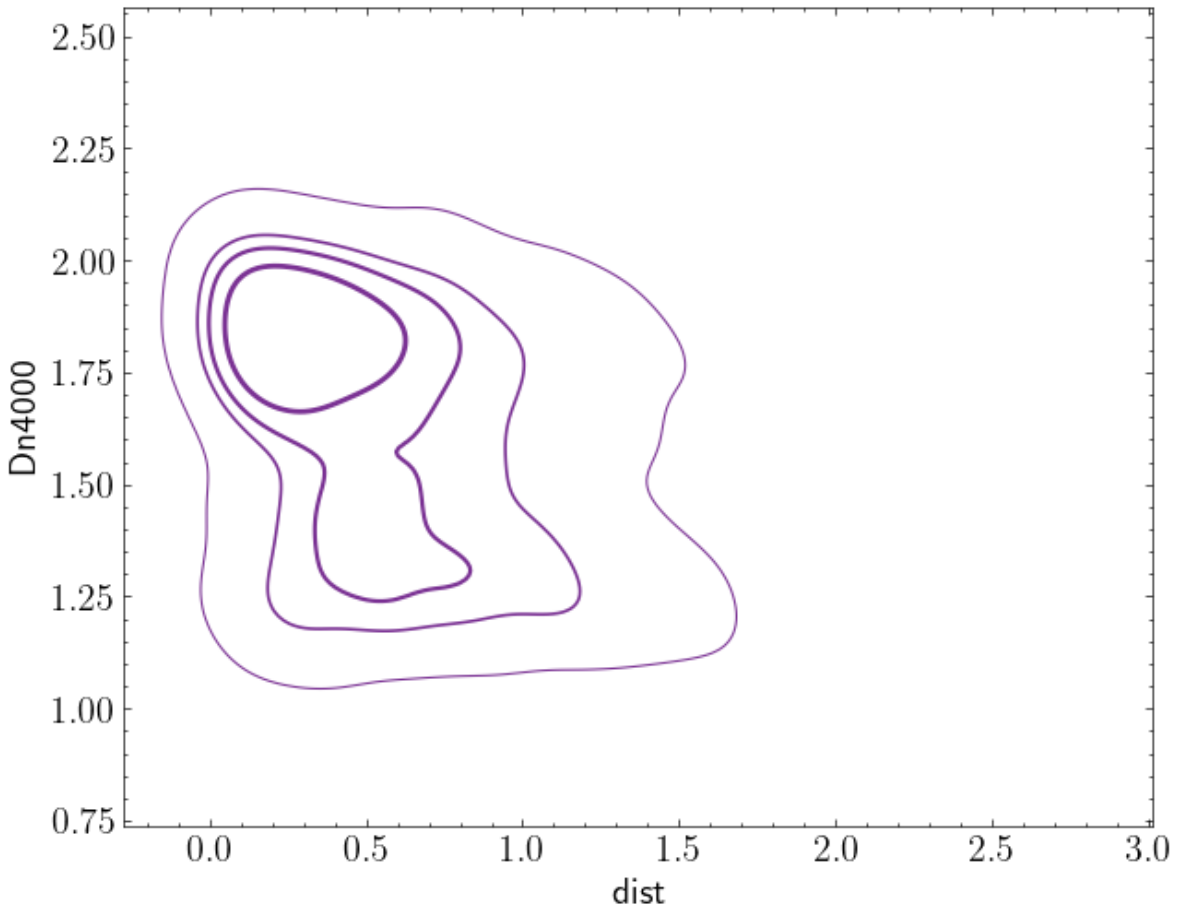
Out[412… <ErrorbarContainer object of 3 artists>



In [276…

```
fig, ax = plt.subplots()

sns.kdeplot(
    ax=ax,
    data=cluster,
    x='dist', y='Dn4000',
    weights='w',
    color=COLORS[CL], levels=LEVELS, linewidths=LWS
)
```

Out[276… <Axes: xlabel='dist', ylabel='Dn4000'>



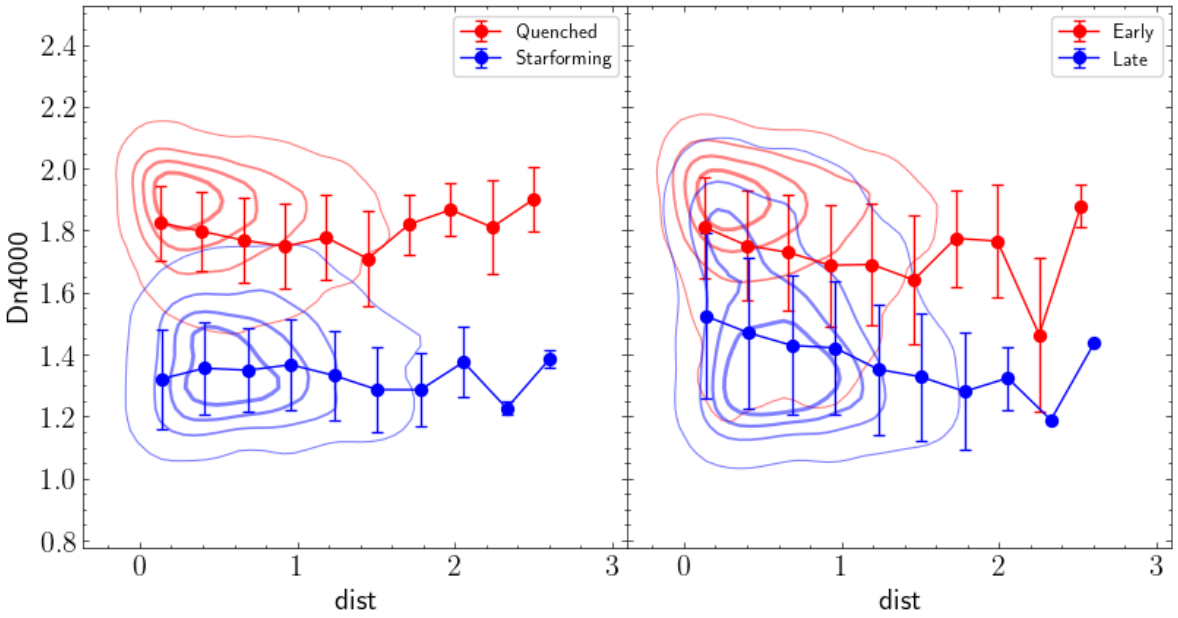
In [535…
```
is_quenched = (cluster['ssfr'] + np.log10(cosmo.age(cluster['z']).to('yr'
fraction_quenched = {
    'Quenched': cluster[is_quenched],
    'Starforming': cluster[~is_quenched]
}
```

In [536…
```
is_early = cluster['C']>=2.5
fraction_type = {
    'Early': cluster[is_early],
    'Late': cluster[~is_early]
}
```

In [537…
```
is_quen_and_early = is_quenched&is_early
fraction_quench_early = {
    'Q+Early': cluster[is_quen_and_early],
    'SF+Late': cluster[~is_quen_and_early]
}
```

In [413…
```
fig, axes = plt.subplots(1,2, figsize=(10,5), sharex=True, sharey=True)

col_median = 'dist'
col_corr = 'Dn4000'
nbins = 10
for i, frac in enumerate([fraction_quenched, fraction_type]):
    for t,m in frac.items():
        bin_center, median, std = weighted_mean_of_col(m, nbins=10, col_m

        axes[i].errorbar(
            bin_center, median, yerr=std,
            c='r' if (t=='Quenched' or t=='Early') else 'b',
            fmt='o-', label=t,
```

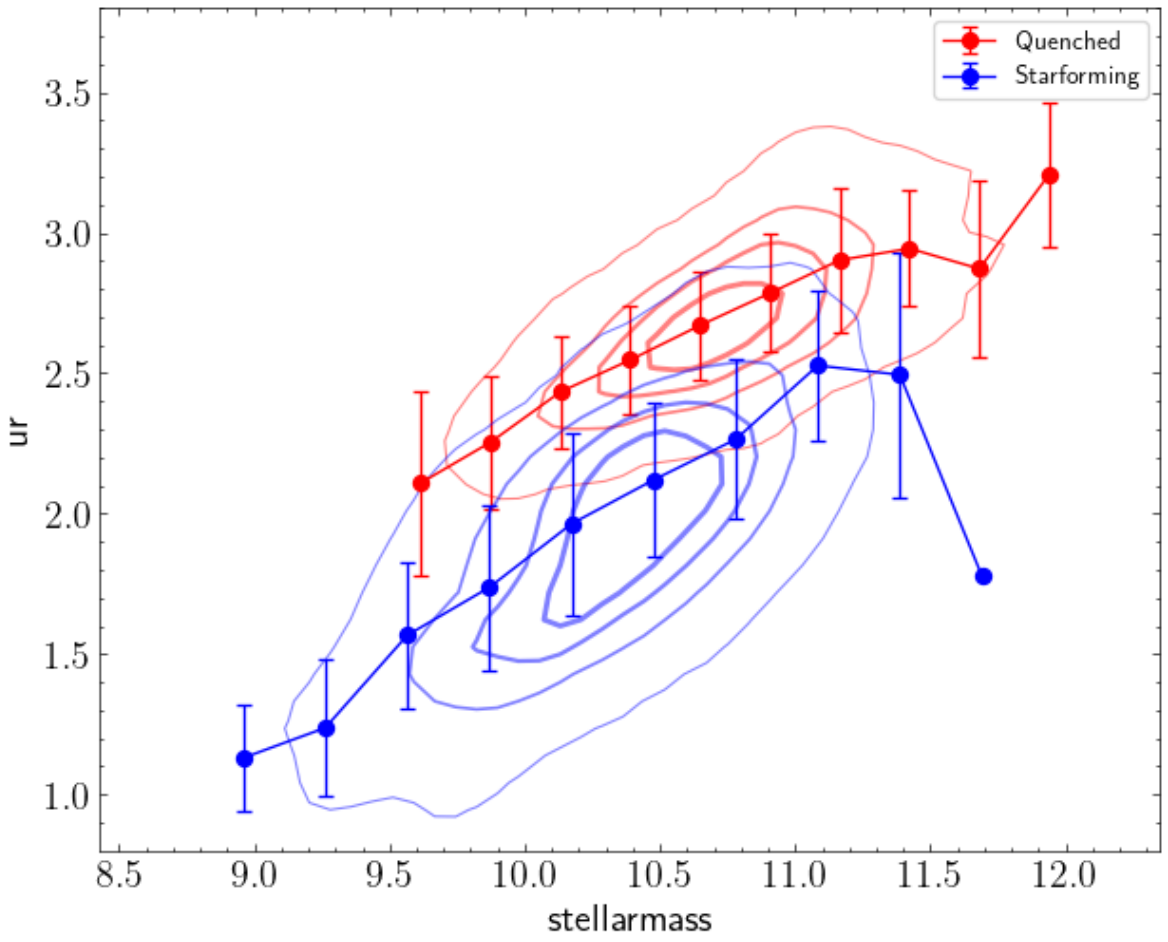
```
        )

        sns.kdeplot(
            ax=axes[i],
            data=m,
            x=col_median, y=col_corr,
            color='r' if (t=='Quenched' or t=='Early') else 'b',
            gridsize=GRID, levels=LEVELS, linewidths=LWS, alpha=0.5
        )

    axes[i].legend()

fig.subplots_adjust(wspace=0)
```



In [414…

```
fig, axes = plt.subplots()

col_median = 'stellarmass'
col_corr = 'ur'
nbins = 10
for t,m in fraction_quenched.items():
    bin_center, median, std = weighted_mean_of_col(m, nbins=10, col_media

    axes.errorbar(
        bin_center, median, yerr=std,
        c='r' if t=='Quenched' else 'b',
        fmt='o-', label=t,
    )

    sns.kdeplot(
        ax=axes,
        data=m,
        x=col_median, y=col_corr,
        color='r' if t=='Quenched' else 'b',
        gridsize=GRID, levels=LEVELS, linewidths=LWS, alpha=0.5
    )

axes.legend()
axes.set_ylim(0.8,3.8)
```

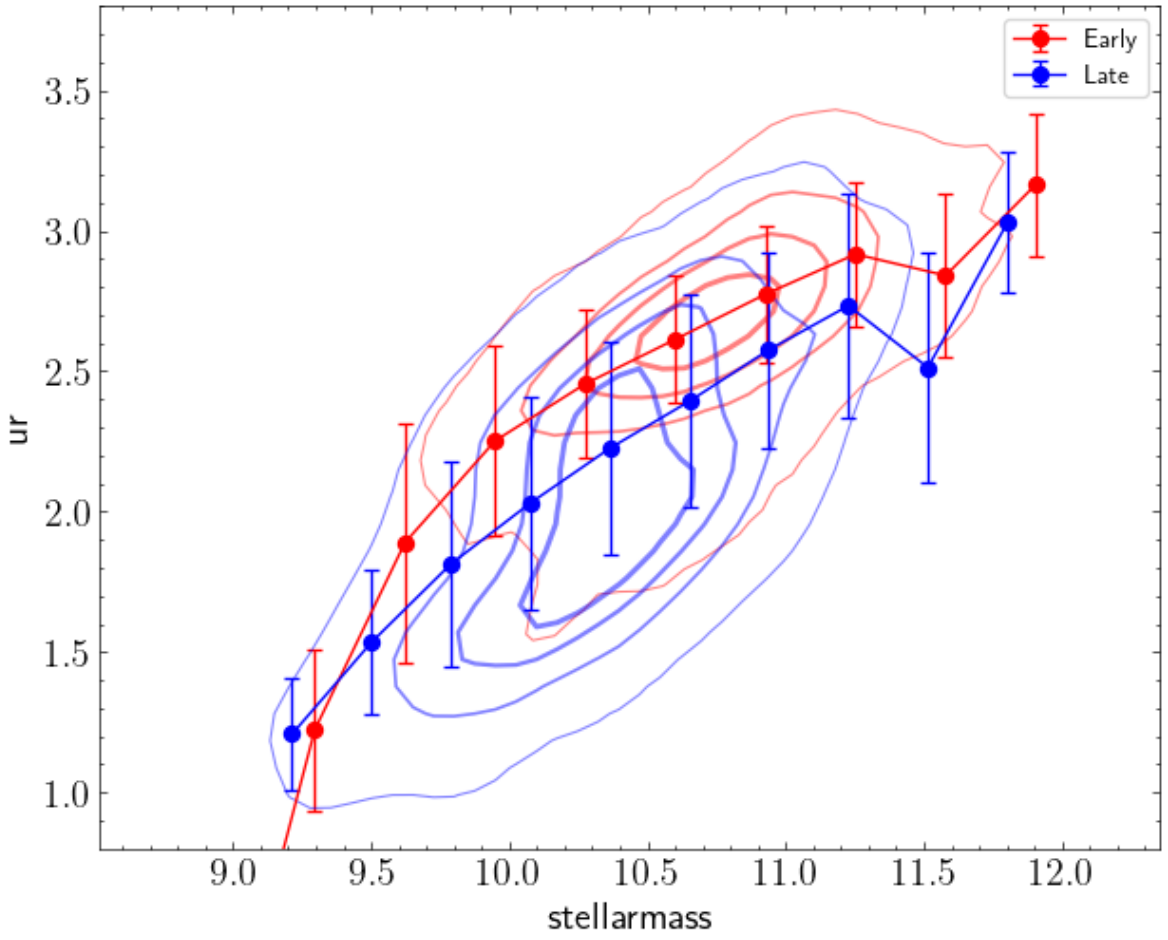
Out[414… (0.8, 3.8)



In [415…

```
fig, axes = plt.subplots()

col_median = 'stellarmass'
col_corr = 'ur'
nbins = 10
for t,m in fraction_type.items():
    bin_center, median, std = weighted_mean_of_col(m, nbins=10, col_media

    axes.errorbar(
        bin_center, median, yerr=std,
        c='r' if t=='Early' else 'b',
        fmt='o-', label=t,
    )

    sns.kdeplot(
        ax=axes,
        data=m,
        x=col_median, y=col_corr,
        color='r' if t=='Early' else 'b',
        gridsize=GRID, levels=LEVELS, linewidths=LWS, alpha=0.5
    )

axes.legend()
axes.set_ylim(0.8,3.8)
```

Out[415… (0.8, 3.8)

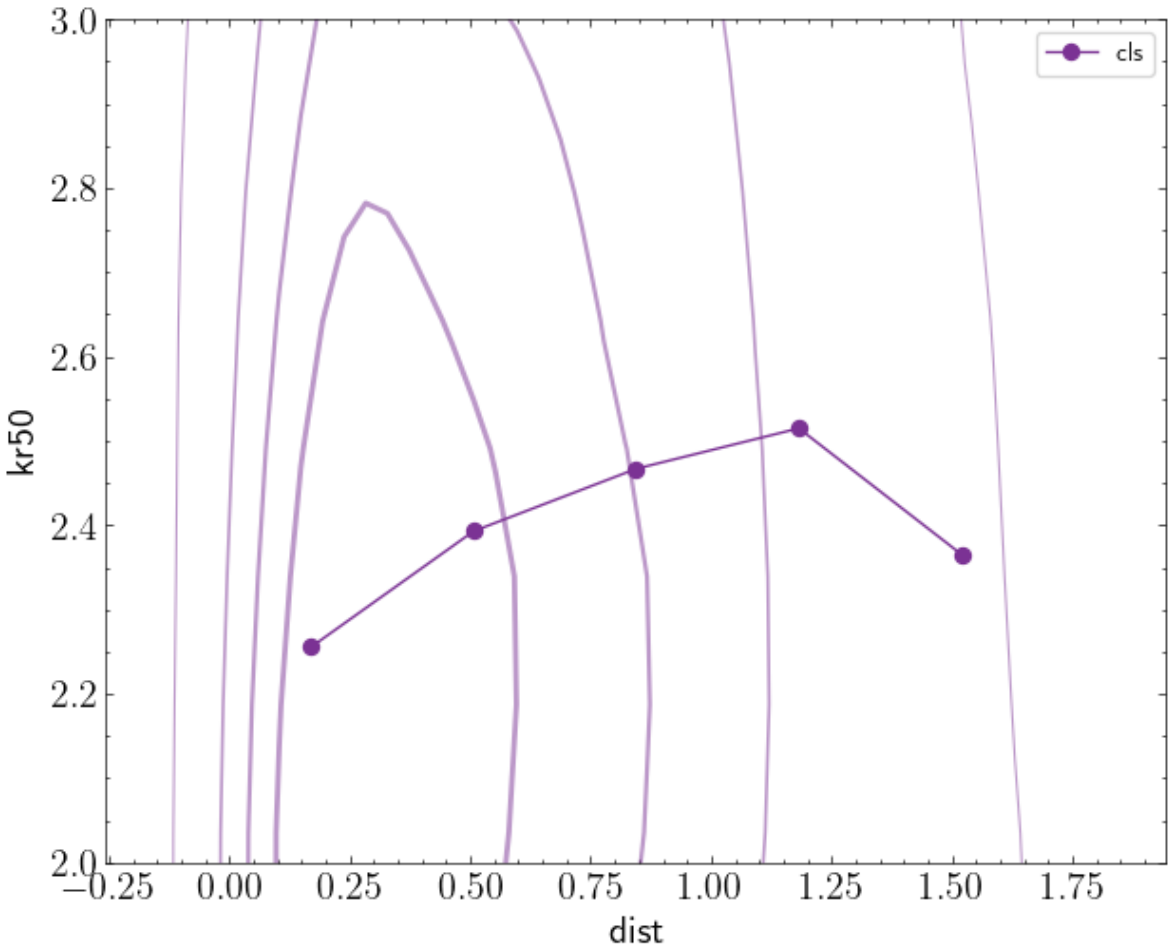


In [565…

```
fig, axes = plt.subplots()

col_median = 'dist'
col_corr = 'kr50'
nbins = 10

mm = cluster.query('kr50<7.0 and dist<1.7')
bin_center, median, std = median_of_col(mm, nbins=5, col_median=col_media
axes.plot(
    bin_center, median, 'o-',
    c=COLORS[CL],
    label=t,
)

sns.kdeplot(
    ax=axes,
    data=mm,
    x=col_median, y=col_corr,
    c=COLORS[CL],
    gridsize=GRID, levels=LEVELS, linewidths=LWS, alpha=0.5
)

axes.legend()
#axes.set_xlim(0.0,1.4)
axes.set_ylim(2.,3.)
#axes.set_yscale('log')
```

```
/home/franco/miniconda3/envs/envpy/lib/python3.9/site-packages/seaborn/dis
tributions.py:1176: UserWarning: The following kwargs were not used by con
tour: 'c'
  cset = contour_func(
```

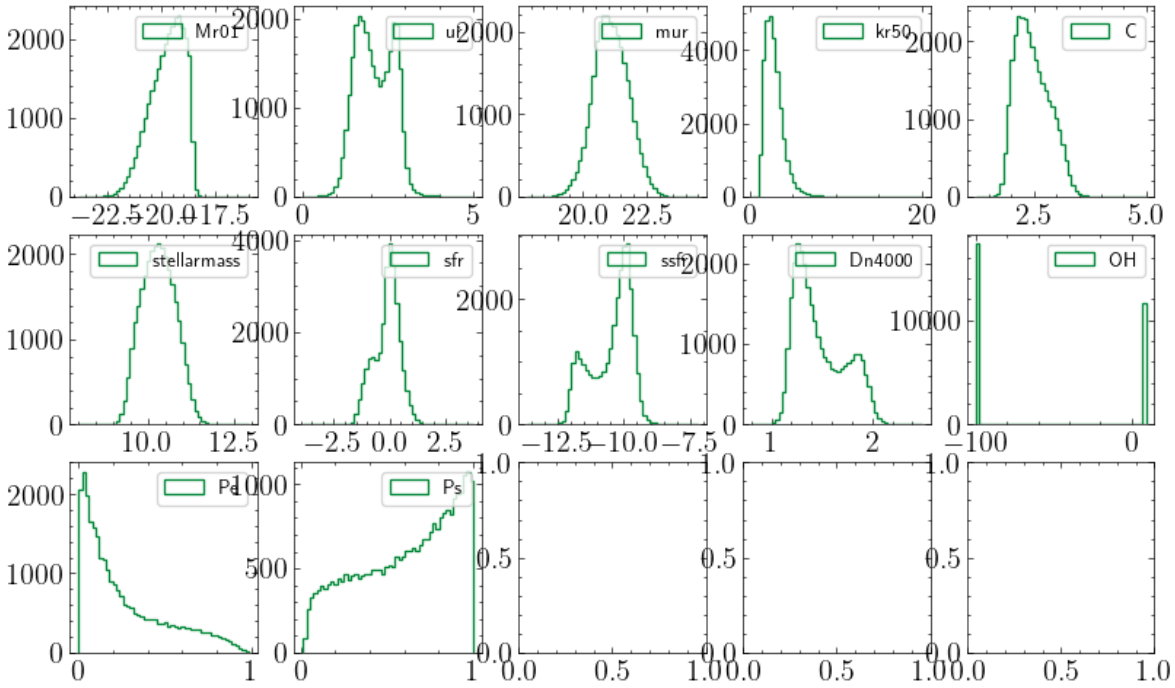
Out[565… (2.0, 3.0)



Propiedades de Campo (?)

In [12]:
```
field = samples[FD]
```

In [13]:
```
# Distribuciones de las columnas, pesadas por 'w'
fig, axes = plt.subplots(3,5,figsize=(10,6))
ax = axes.flatten()
i=0
for col in COLUMNS[FD]:
    if col=='ra': continue
    if col=='dec': continue
    if col=='z': continue
    if col=='w': continue
    ax[i].hist(field[col], bins=50, weights=field['w'], histtype='step',
    ax[i].legend()
    i+=1
```

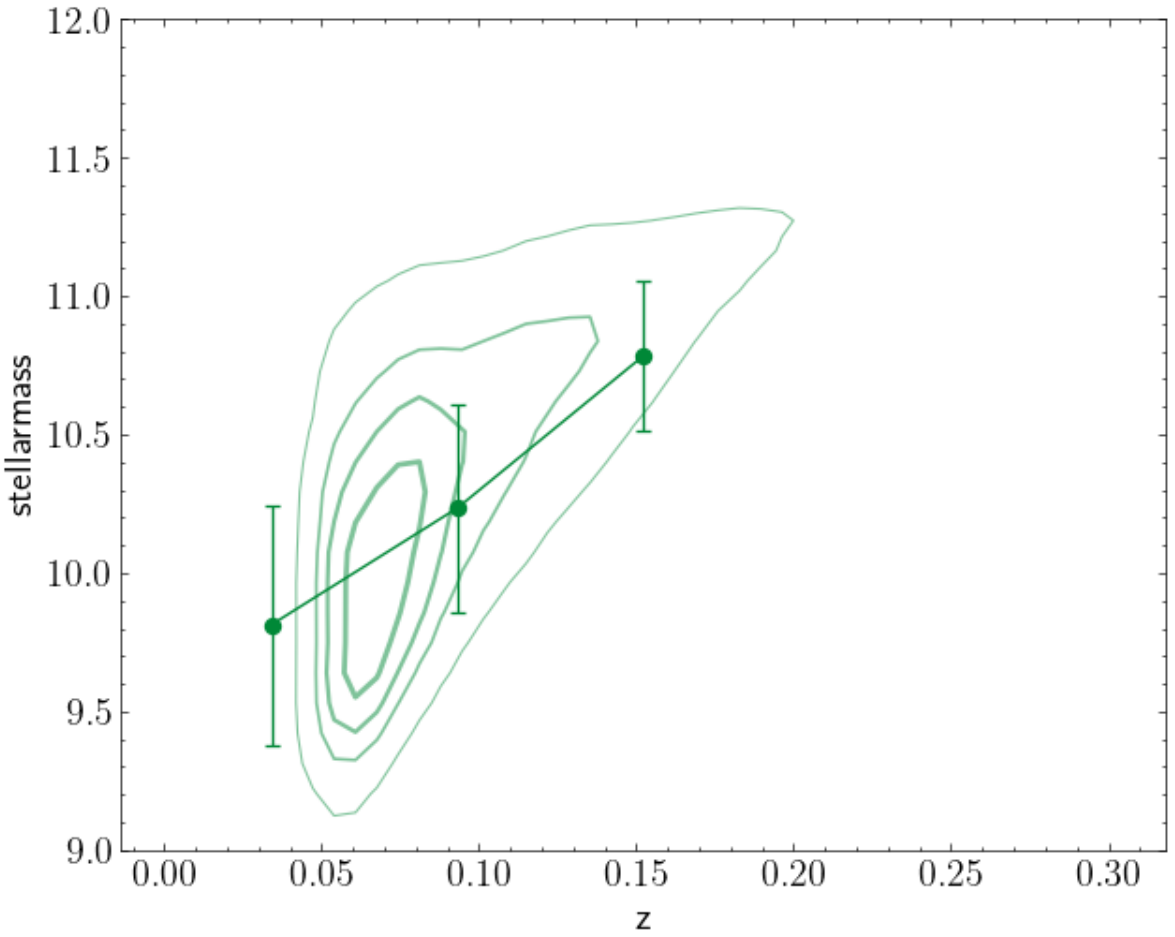


In [421…

```
fig, ax = plt.subplots()

sns.kdeplot(
    ax=ax,
    data=field,
    x='z', y='stellarmass',
    weights='w',
    color=COLORS[FD], alpha=0.5,
    gridsize=GRID, levels=LEVELS, linewidths=LWS,
)

bin_center, median, std = weighted_mean_of_col(field, nbins=5, col_median
median[median==0.0] = np.nan
ax.errorbar(
    bin_center, median, yerr=std,
    c=COLORS[FD],
    fmt='o-',
)

ax.set_ylim(9.0, 12.0)
```

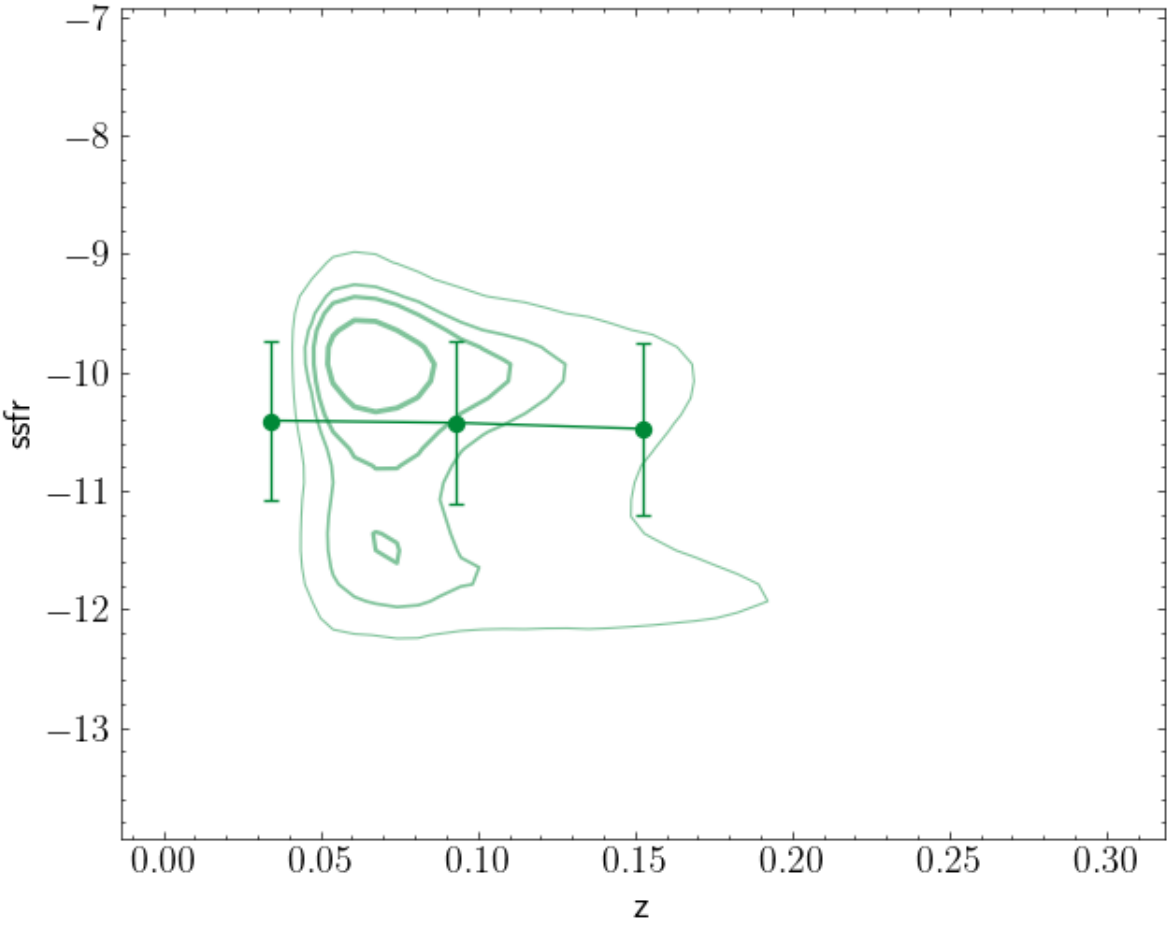
Out[421…

```
(9.0, 12.0)
```



In [425…
```
fig, ax = plt.subplots()

sns.kdeplot(
    ax=ax,
    data=field,
    x='z', y='ssfr',
    weights='w',
    color=COLORS[FD], alpha=0.5,
    gridsize=GRID, levels=LEVELS, linewidths=LWS,
)
```

Out[425…
```
<ErrorbarContainer object of 3 artists>
```



In [434…
```
is_quenched = (field['ssfr'] + np.log10(cosmo.age(field['z']).to('yr').va
fraction_quenched = {
    'Quenched': field[is_quenched],
    'Starforming': field[~is_quenched]
}
```

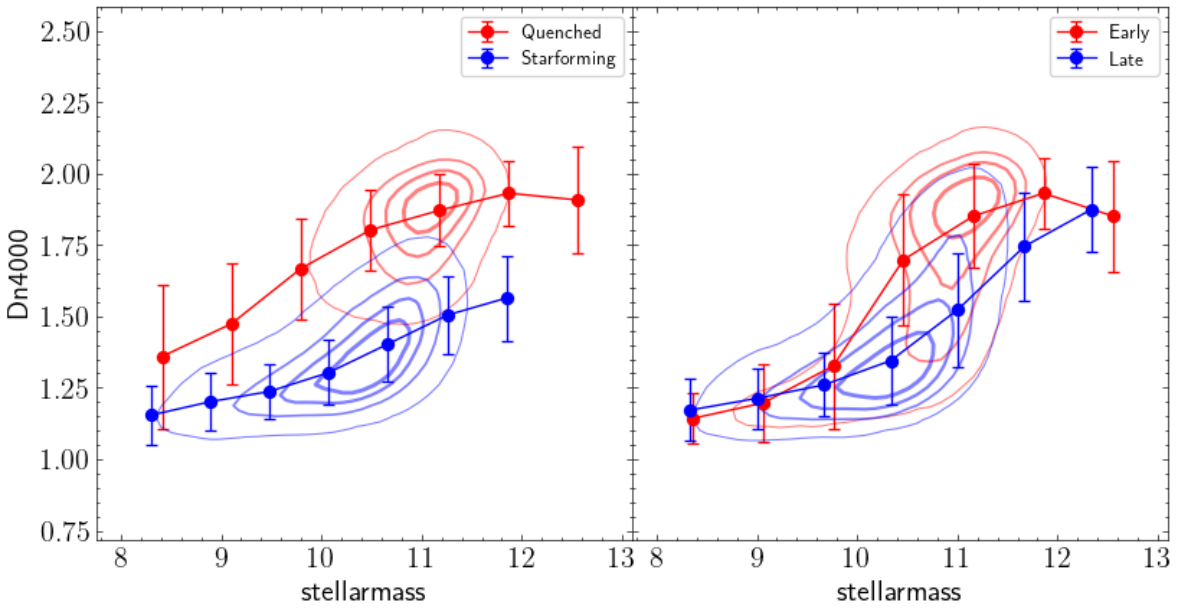
```
219483
-252885
```

In [429…
```
is_early = field['C']>=2.5
fraction_type = {
    'Early': field[is_early],
    'Late': field[~is_early]
}
```

In [430…
```
is_quen_and_early = is_quenched&is_early
fraction_quench_early = {
    'Q+Early': field[is_quen_and_early],
    'SF+Late': field[~is_quen_and_early]
}
```

In [482…
```
fig, axes = plt.subplots(1,2, figsize=(10,5), sharex=True, sharey=True)

col_median = 'stellarmass'
col_corr = 'Dn4000'
nbins = 10
for i, frac in enumerate([fraction_quenched, fraction_type]):
    for t,m in frac.items():
        bin_center, median, std = median_of_col(m, nbins=7, col_median=co

        axes[i].errorbar(
```

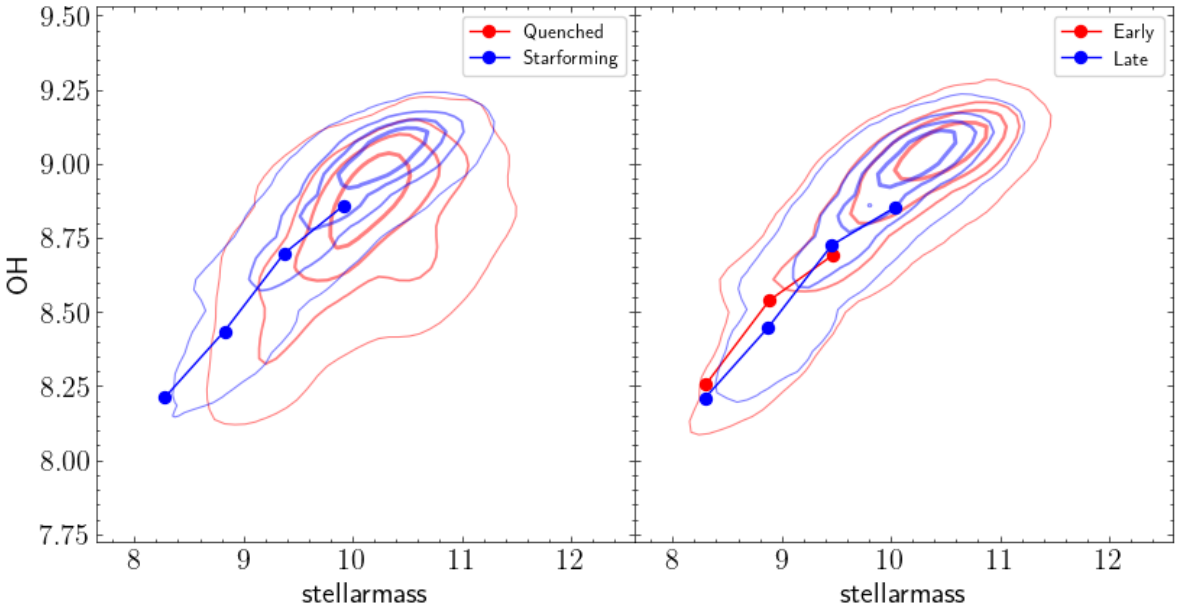
```
            bin_center, median, yerr=std,
            c='r' if (t=='Quenched' or t=='Early') else 'b',
            fmt='o-', label=t,
        )

        sns.kdeplot(
            ax=axes[i],
            data=m,
            x=col_median, y=col_corr,
            color='r' if (t=='Quenched' or t=='Early') else 'b',
            gridsize=GRID, levels=LEVELS, linewidths=LWS, alpha=0.5
        )

    axes[i].legend()

fig.subplots_adjust(wspace=0)
```



In [464…

```
fig, axes = plt.subplots(1,2, figsize=(10,5), sharex=True, sharey=True)

col_median = 'stellarmass'
col_corr = 'OH'
nbins = 10
mask = 'OH>-1'
for i, frac in enumerate([fraction_quenched, fraction_type]):
    for t,m in frac.items():
        bin_center, median, std = median_of_col(m.query(mask), nbins=7, c
        median[median<=-1] = np.nan
        #axes[i].errorbar(
        #    bin_center, median, yerr=std,
        #    c='r' if (t=='Quenched' or t=='Early') else 'b',
        #    fmt='o-', label=t,
        #)
        axes[i].plot(
            bin_center, median, 'o-',
            c='r' if (t=='Quenched' or t=='Early') else 'b',
            label=t,
        )
        sns.kdeplot(
            ax=axes[i],
            data=m.query(mask),
            x=col_median, y=col_corr,
```

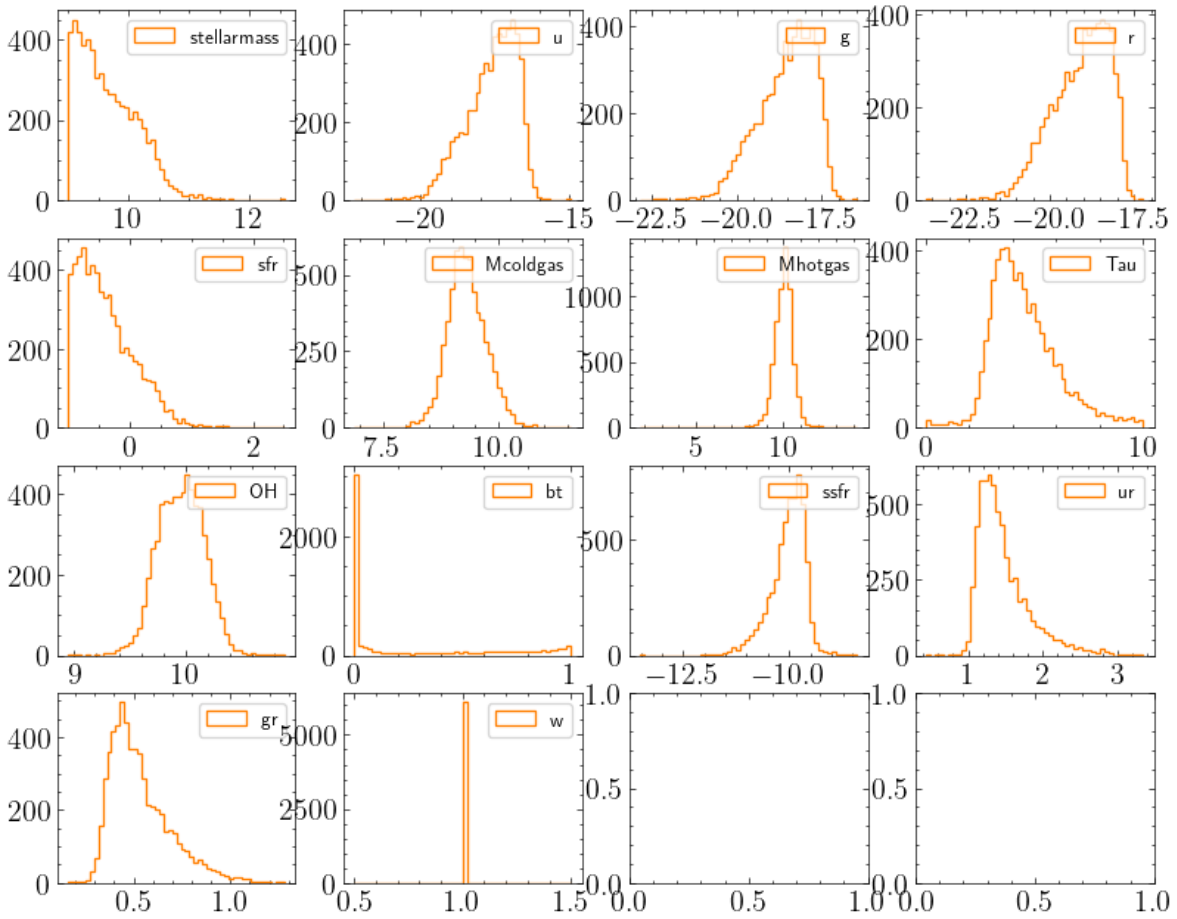
```
            color='r' if (t=='Quenched' or t=='Early') else 'b',
            gridsize=GRID, levels=LEVELS, linewidths=LWS, alpha=0.5
        )

    axes[i].legend()

fig.subplots_adjust(wspace=0)
#axes[0].set_ylim(0.0, 10)
```



Propiedades Semianalítico

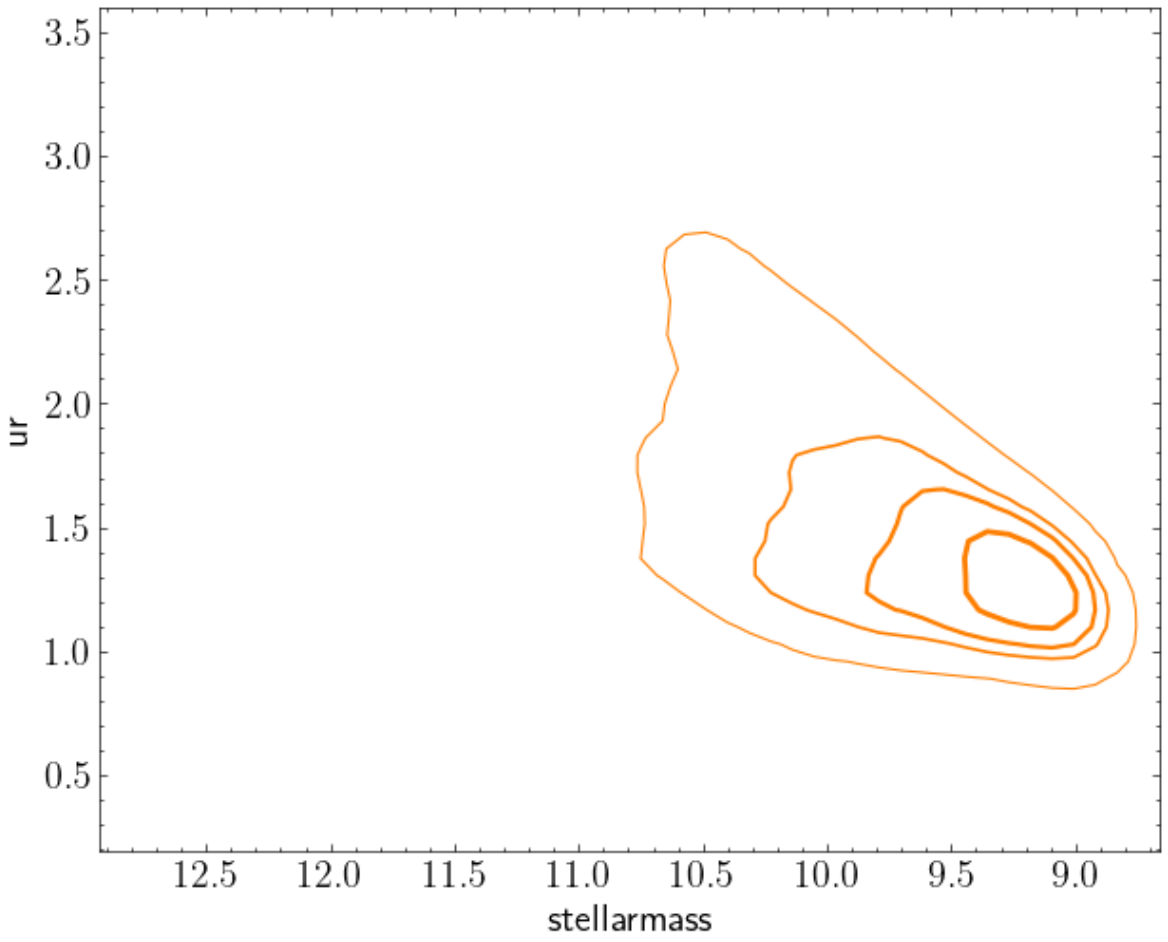
In [14]:
```
semia = samples[SA]
```

In [15]:
```
# Distribuciones de las columnas
fig, axes = plt.subplots(4,4,figsize=(10,8))
ax = axes.flatten()
i=0
for col in COLUMNS[SA]:
    if col=='cls': continue
    if col=='ig': continue
    if col=='type': continue
    if col=='clase': continue
    ax[i].hist(semia[col], bins=50, histtype='step', color=COLORS[SA], la
    ax[i].legend()
    i+=1
```



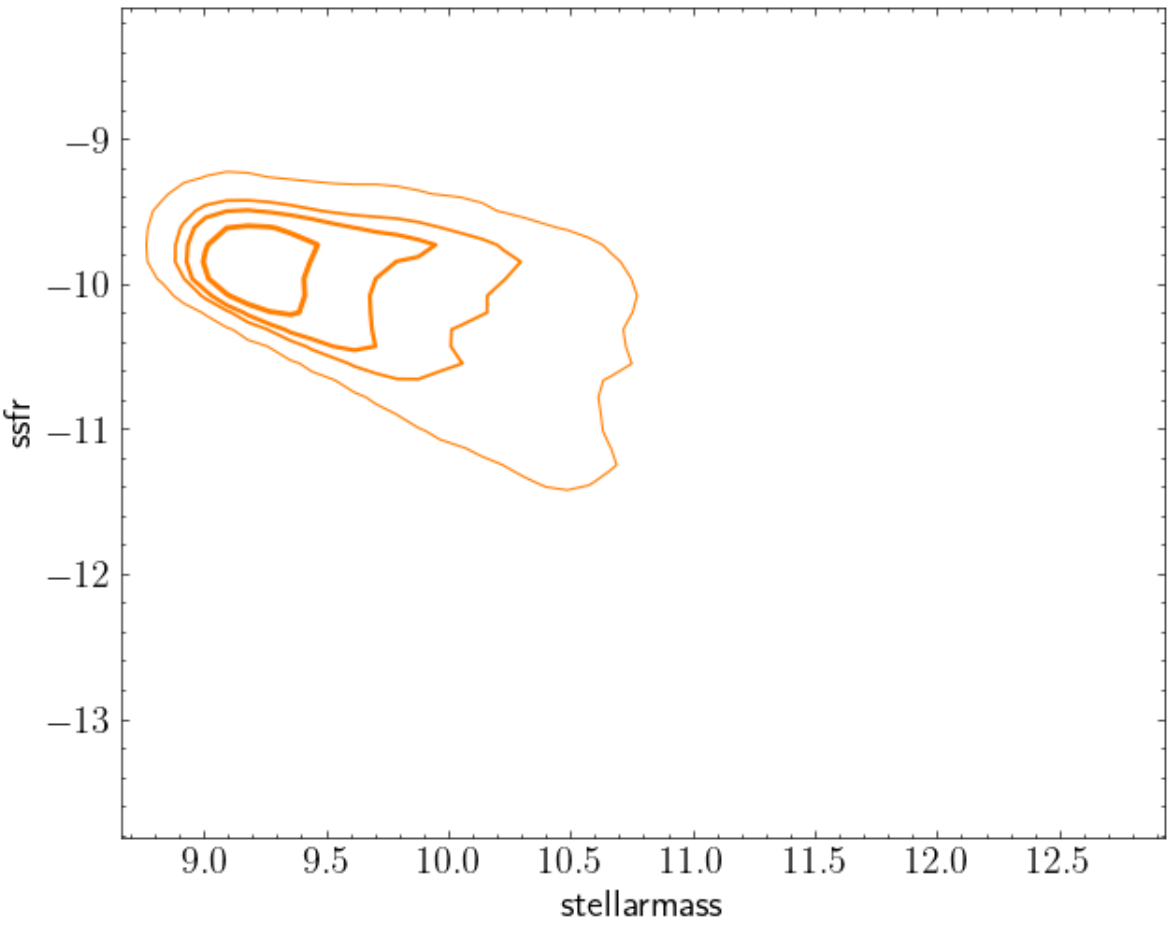
In [502…

```
fig, ax = plt.subplots()

sns.kdeplot(
    ax=ax,
    data=semia,
    x='stellarmass', y='ur',
    weights='w',
    color=COLORS[SA], gridsize=GRID, levels=LEVELS, linewidths=LWS
)

ax.invert_xaxis()
```



In [454…
```
fig, ax = plt.subplots()

sns.kdeplot(
    ax=ax,
    data=semia,
    x='stellarmass', y='ssfr',
    weights='w',
    color=COLORS[SA], gridsize=GRID, levels=LEVELS, linewidths=LWS
)
```

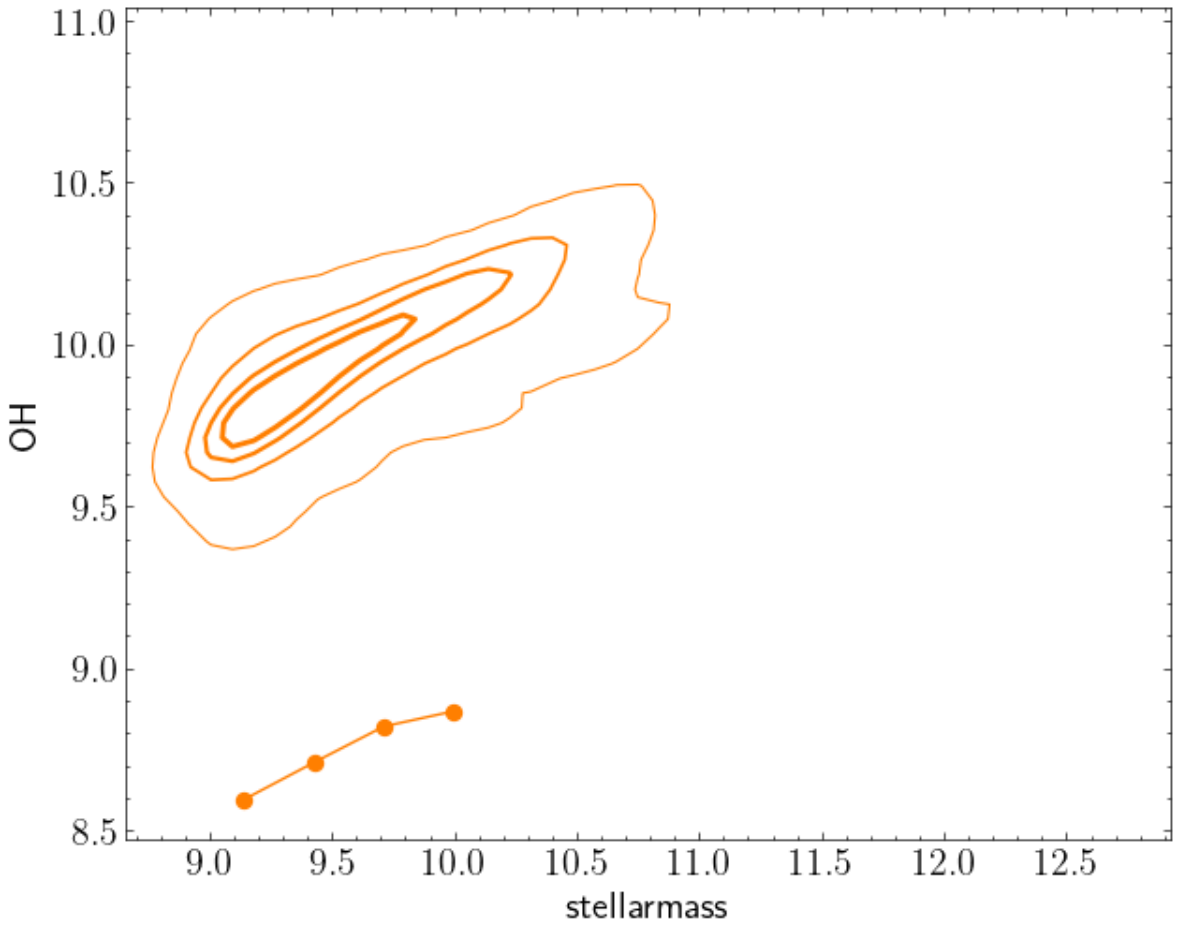
Out[454… `<Axes: xlabel='stellarmass', ylabel='ssfr'>`



In [499…

```
fig, ax = plt.subplots()

semia_cut = semia.query('stellarmass < 11.0')
bin_center, median, std = median_of_col(semia_cut, nbins=7, col_median='s
median[median<0.0]=np.nan
ax.plot(
    bin_center, median, 'o-',
    color=COLORS[SA],
)

sns.kdeplot(
    ax=ax,
    data=semia,
    x='stellarmass', y='OH',
    weights='w',
    color=COLORS[SA], gridsize=GRID, levels=LEVELS, linewidths=LWS
)
```

Out[499… `<Axes: xlabel='stellarmass', ylabel='OH'>`



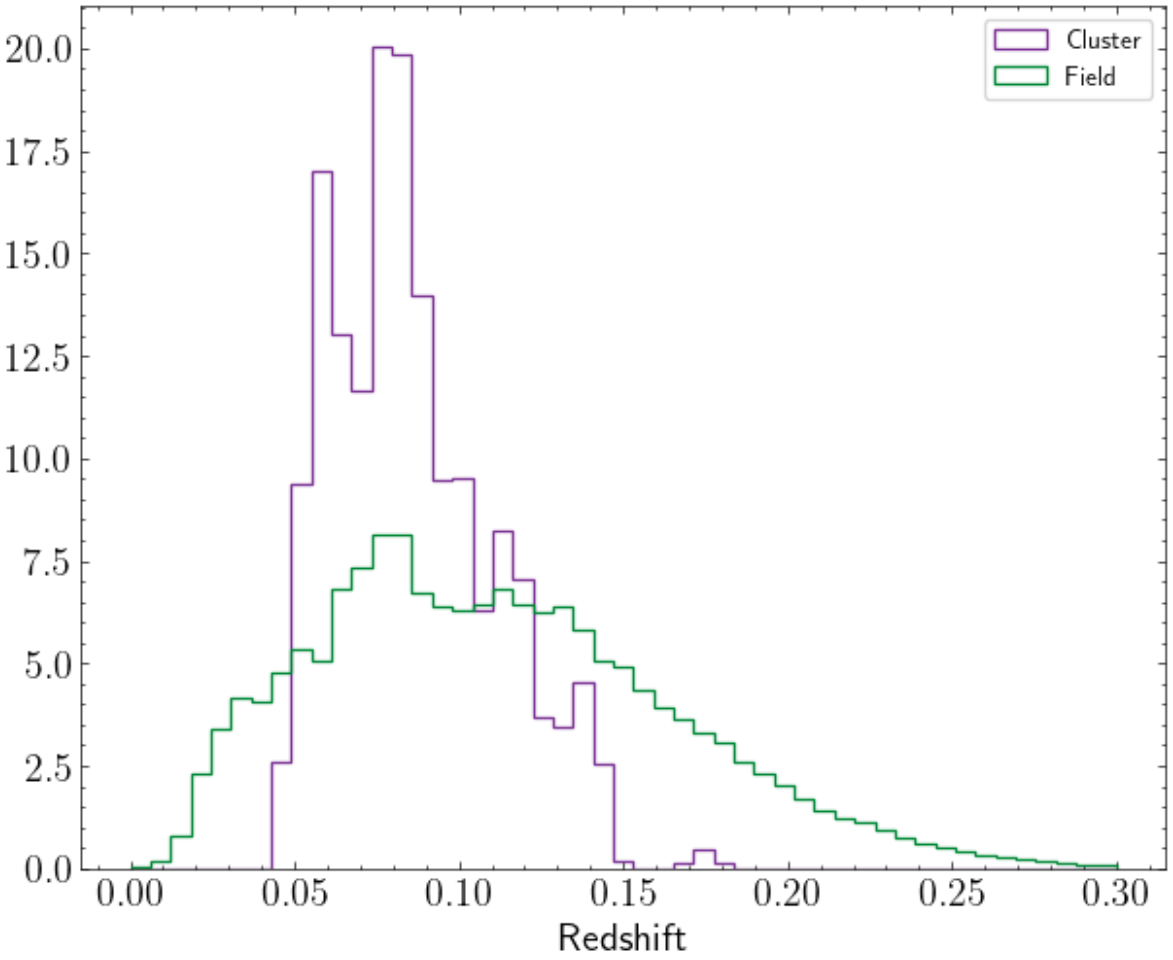
Comparación cluster-campo

Cluster y campo, semianalitico no tiene redshift

Distribución de redshift

In [513…
```
bins = np.linspace(0.0, 0.3, 50)

fig, ax = plt.subplots()
for n in [CL, FD]:
    ax.hist(
        samples[n]['z'],
        bins=bins,
        density=True,
        histtype='step',
        label=n,
        color=COLORS[n]
    )
ax.legend()
ax.set_xlabel('Redshift')
```

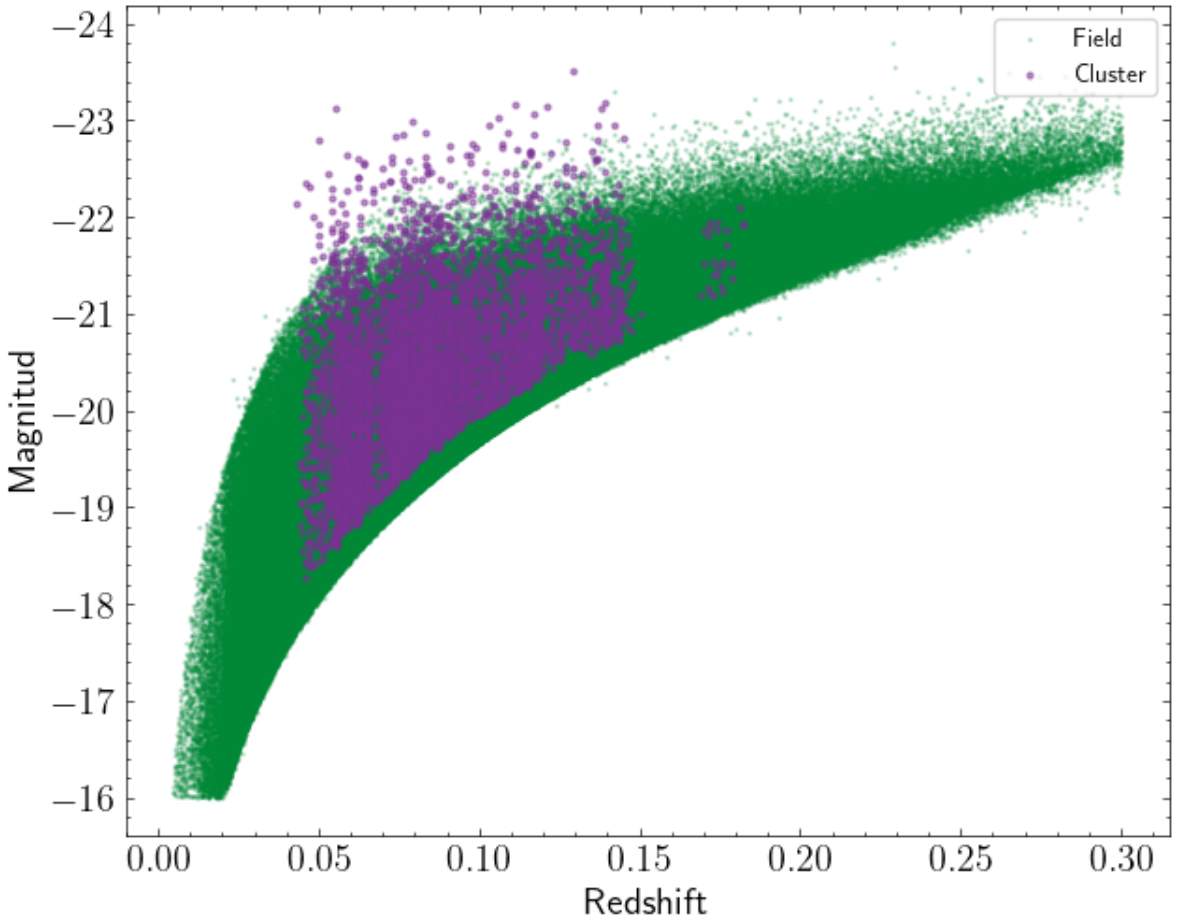
Out[513… `Text(0.5, 0, 'Redshift')`



Completitud

In [514…
```
fig, ax = plt.subplots()

for i,n in enumerate([FD, CL]):
    ax.scatter(
        samples[n]['z'], samples[n]['Mr01'],
        s=1 if n==FD else 5, alpha=0.2 if n==FD else 0.5, label=n, color=
    )

ax.set_xlabel('Redshift')
ax.set_ylabel('Magnitud')
ax.invert_yaxis()
ax.legend()
```

Out[514… `<matplotlib.legend.Legend at 0x7c994f852a00>`



Vemos que los catalogos de campo y cluster no son comparables, se debe realizar un corte en redshift para poder tomar muestras compatibles.

Para eso, se toman redshfits entre $0.05 \leq z \leq 0.14$, que son aproximadamente el rango donde tenemos clusters. Fuera del rango, sólo se tienen galaxias de campo.

Además se utilizará una muestra completa por flujo, que implica pesar por $1/V_{\max}$ en las siguientes figuras.

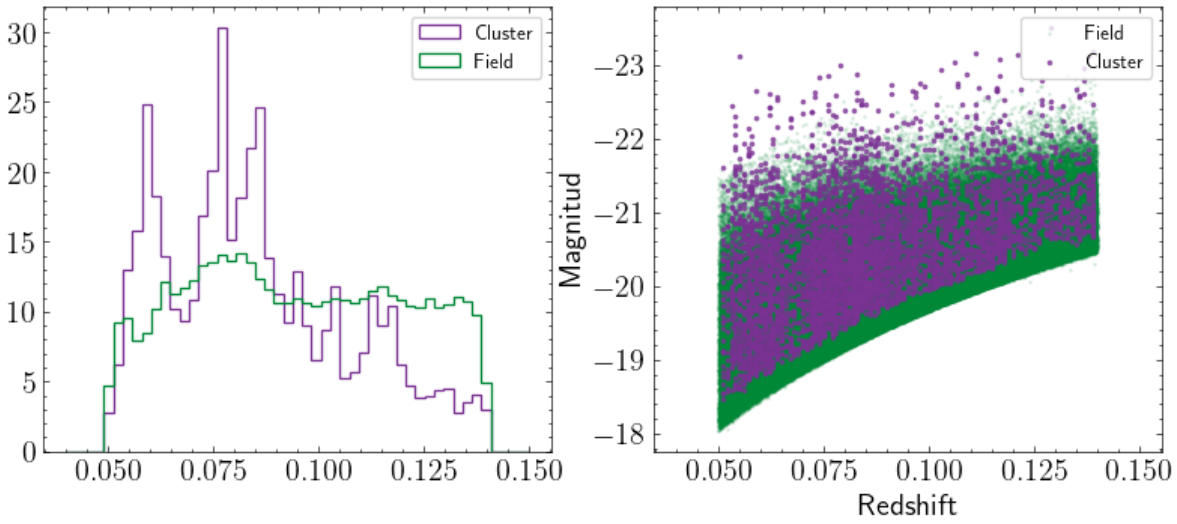
In [515…
```
z_cut = 'z > 0.05 and z < 0.14'
```

In [516…
```
S = {} # Muestra que utilizaremos de aquí en adelante
for n in [CL,FD]:
    S[n] = samples[n].query(z_cut, inplace=False)
S[SA] = samples[SA]
```

In [517…
```
bins = np.linspace(0.04, 0.15, 50)
fig, (ax1, ax2) = plt.subplots(1,2,figsize=(10,4),sharex=True)

for n in [CL, FD]:
    ax1.hist(
        S[n]['z'],
        bins=bins,
        density=True,
        histtype='step',
        label=n,
        color=COLORS[n]
    )
ax1.legend()
```

```
for i,n in enumerate([FD, CL]):
    ax2.scatter(
        S[n]['z'], S[n]['Mr01'],
        s=1 if n==FD else 3, alpha=0.1 if n==FD else 0.7,
        label=n, color=COLORS[n]
    )

ax2.set_xlabel('Redshift')
ax2.set_ylabel('Magnitud')
ax2.invert_yaxis()
ax2.legend()
```

Out[517… <matplotlib.legend.Legend at 0x7c997b5db2b0>



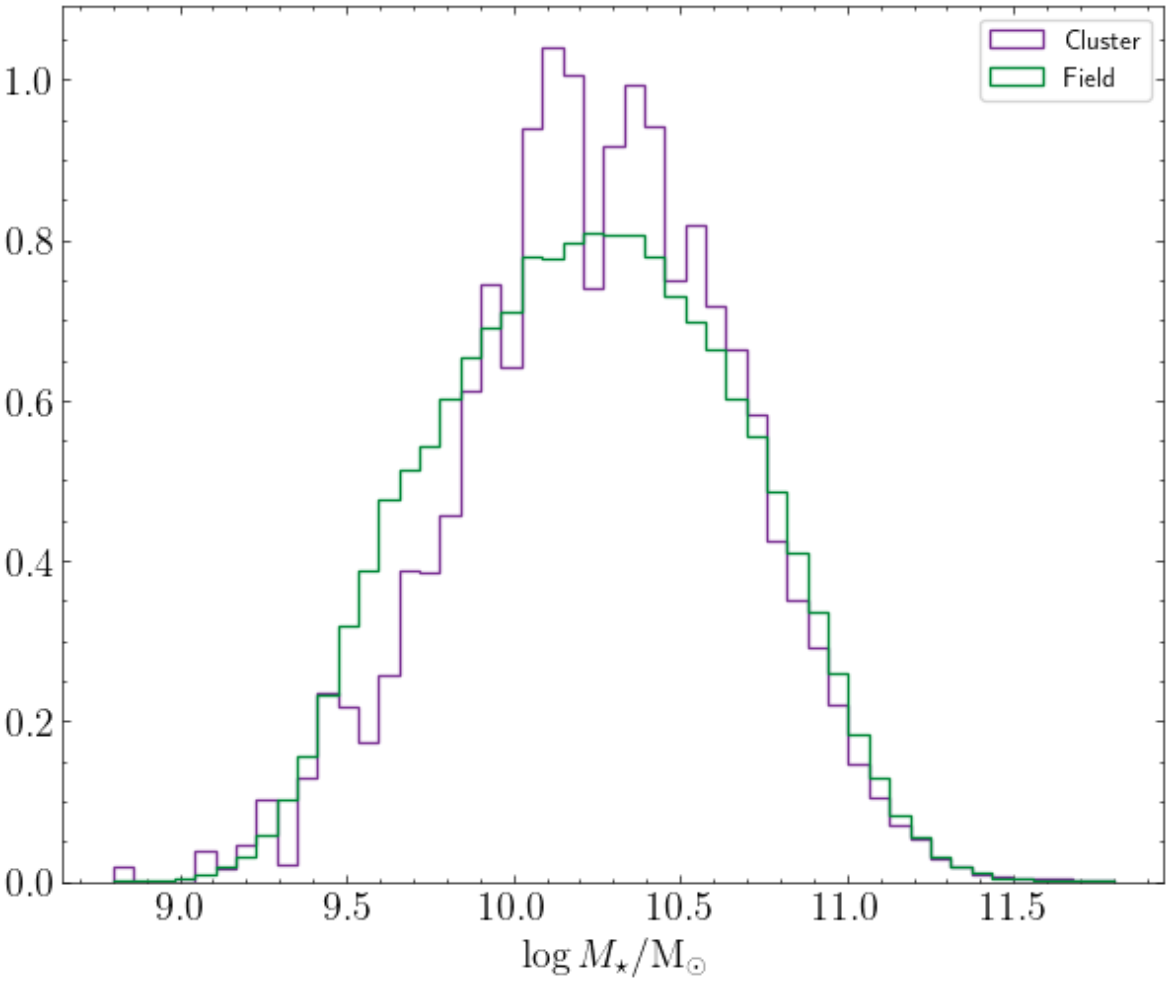
Distribución de masa estelar $M_\star$

Son comparables los dos catálogos?

In [518…

```
bins = np.linspace(8.8, 11.8, 50)
fig, ax = plt.subplots()

for n in [CL,FD]:
    ax.hist(
        S[n]['stellarmass'],
        weights=S[n]['w'],
        bins=bins,
        density=True,
        histtype='step',
        label=n,
        color=COLORS[n]
    )

ax.set_xlabel('$\log M_{\star}/\mathrm{M}_{\odot}$')
ax.legend();

# Son muestras similares?
ks_res = ks_2samp(S[CL]['stellarmass'], S[FD]['stellarmass'], method='exa
print(f'pvalue = {ks_res.pvalue:2.2e}')
if ks_res.pvalue > 0.05:
    print('La información no es suficiente para rechazar la hip. nula')
else:
    print('Las muestras son diferentes')
```

```
pvalue = 5.01e-17
Las muestras son diferentes
```



El KS-test muestra que las distribuciones de masa estelar son diferentes, es decir, las masas de ambos catálogos son diferentes.

De la figura anterior se puede ver que las galaxias de campo tienen un exceso de densidad en masas entre $9.5 < \log M_\star < 10$ respecto a las de cúmulo, mientras que estas ultimas tienen masas en el rango $10 < \log M_\star < 10.5$ de manera más frecuente que las de campo.

Star Formation Rate: Quenched vs Starforming

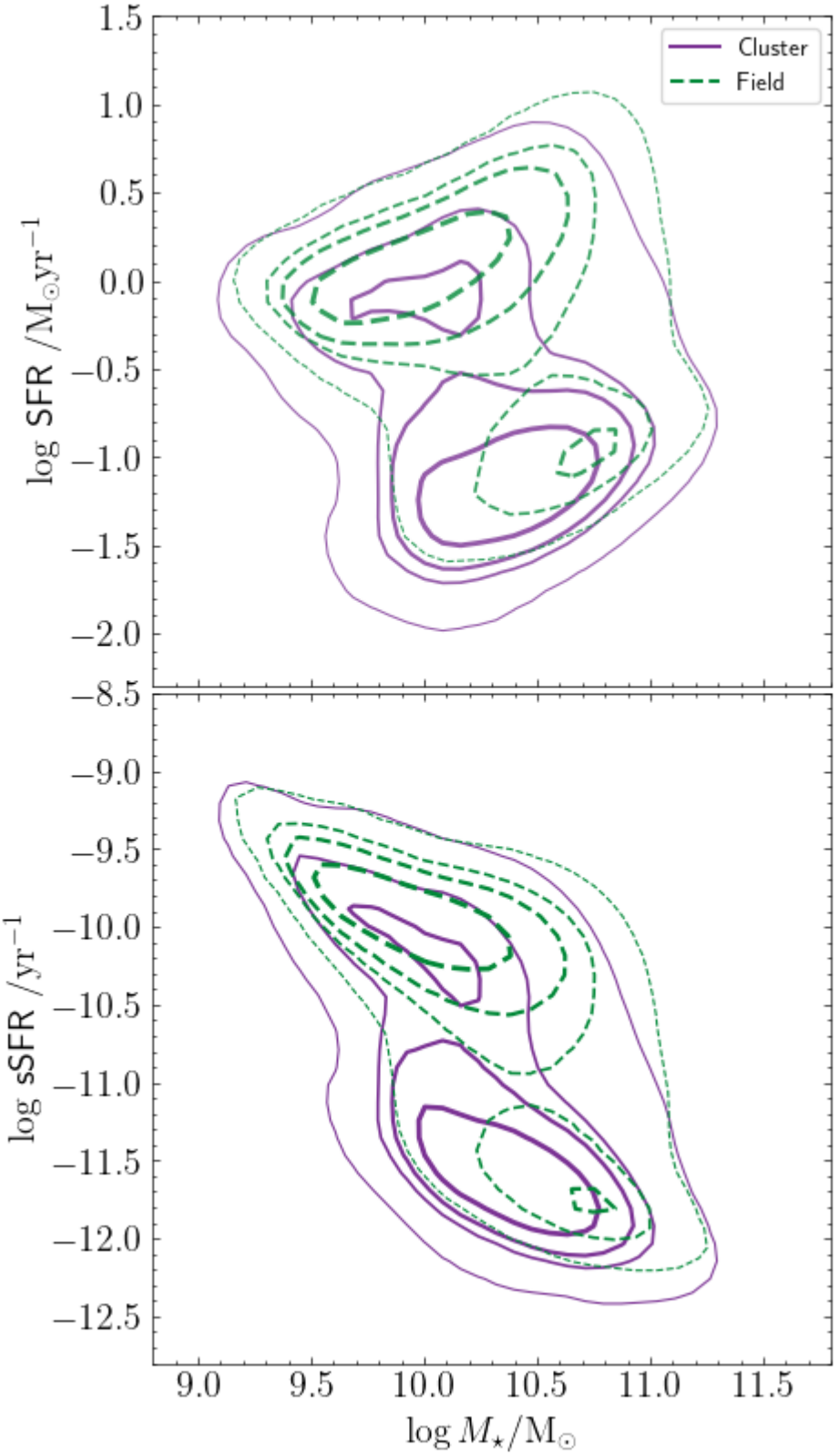
Se dice que una galaxia está apagada (quenched) cuando se cumple

$$\log sSFR + \log t(z) < \log 0.2$$

donde $sSFR$ es su tasa de formación estelar específica, z su redshift y $t(z)$ la edad del Universo a redshift z.

In [519…

```
fig, (ax1, ax2) = plt.subplots(2,1, figsize=(5,10), sharex=True)

for n in [CL, FD]:
    ax1.plot([],[], lw=1.5, c=COLORS[n], label=n, ls='--' if n==FD else '
    sns.kdeplot(
        ax = ax1,
        data=S[n],
        x='stellarmass', y='sfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color=COLORS[n], linestyles='--' if n==FD else '-', alpha=0.8
    )
    sns.kdeplot(
        ax = ax2,
        data=S[n],
        x='stellarmass', y='ssfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color=COLORS[n], linestyles='--' if n==FD else '-', alpha=1.0
    )

ax1.legend()
ax1.set_xlim(8.8, 11.8)
ax1.set_ylim(-2.3, 1.5)
ax2.set_ylim(-12.8, -8.5)

ax2.set_xlabel('$\log M_{\star}/\mathrm{M}_\odot$')
ax1.set_ylabel('$\log$ SFR $/\mathrm{M}_{\odot} \mathrm{yr}^{-1}$')
ax2.set_ylabel('$\log$ sSFR $/\mathrm{yr}^{-1}$')

fig.subplots_adjust(hspace=0.01)
```

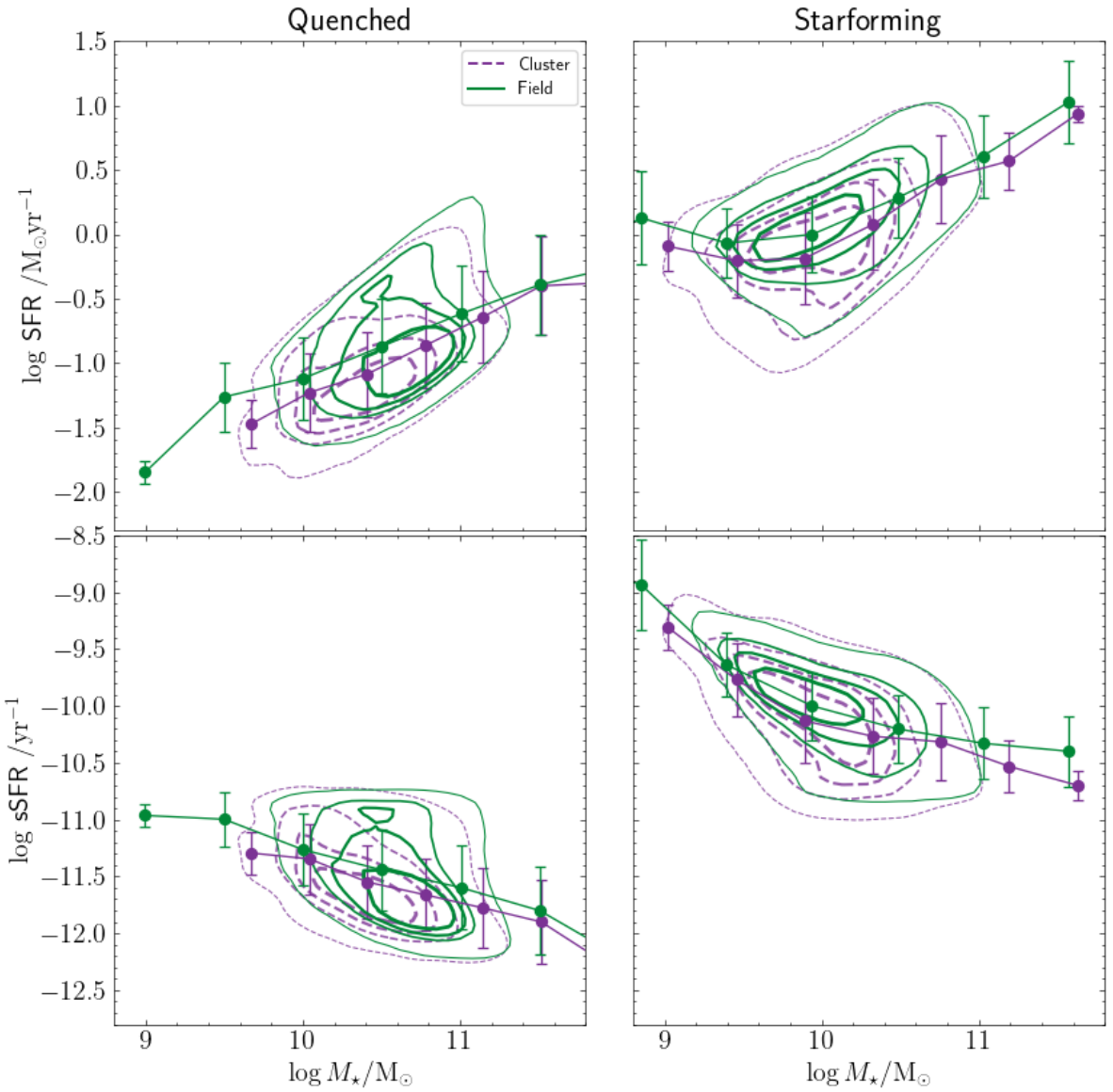


In [524…
```
quenched = {}
starforming = {}
for n in [CL,FD]:
    is_quenched = (S[n]['ssfr'] + np.log10(cosmo.age(S[n]['z']).to('yr').
    quenched[n] = S[n][is_quenched]
    starforming[n] = S[n][~is_quenched]
```

In [533…
```
fig, ((ax11, ax12), (ax21, ax22)) = plt.subplots(2,2, figsize=(10,10), sh

for n in [CL, FD]:
    ax11.plot([],[], lw=1.5, c=COLORS[n], label=n, ls='--' if n==CL else
```

```
    sns.kdeplot(
        ax = ax11,
        data=quenched[n],
        x='stellarmass', y='sfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color=COLORS[n], linestyles='--' if n==CL else '-', alpha=0.8 if
    )
    bines_central, median, std = weighted_mean_of_col(data=quenched[n], n
    ax11.errorbar(
        bines_central, median, yerr=std,
        fmt='o-',
        color=COLORS[n],
    )

    sns.kdeplot(
        ax = ax21,
        data=quenched[n],
        x='stellarmass', y='ssfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color=COLORS[n], linestyles='--' if n==CL else '-', alpha=0.8 if
    )
    bines_central, median, std = weighted_mean_of_col(data=quenched[n], n
    ax21.errorbar(
        bines_central, median, yerr=std,
        fmt='o-',
        color=COLORS[n],
    )

    sns.kdeplot(
        ax = ax12,
        data=starforming[n],
        x='stellarmass', y='sfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color=COLORS[n], linestyles='--' if n==CL else '-', alpha=0.8 if
    )
    bines_central, median, std = weighted_mean_of_col(data=starforming[n]
    ax12.errorbar(
        bines_central, median, yerr=std,
        fmt='o-',
        color=COLORS[n],
    )

    sns.kdeplot(
        ax = ax22,
        data=starforming[n],
        x='stellarmass', y='ssfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color=COLORS[n], linestyles='--' if n==CL else '-', alpha=0.8 if
    )
    bines_central, median, std = weighted_mean_of_col(data=starforming[n]
    ax22.errorbar(
        bines_central, median, yerr=std,
        fmt='o-',
        color=COLORS[n],
    )

ax11.legend()
ax11.set_xlim(8.8, 11.8)
ax11.set_ylim(-2.3, 1.5)
```

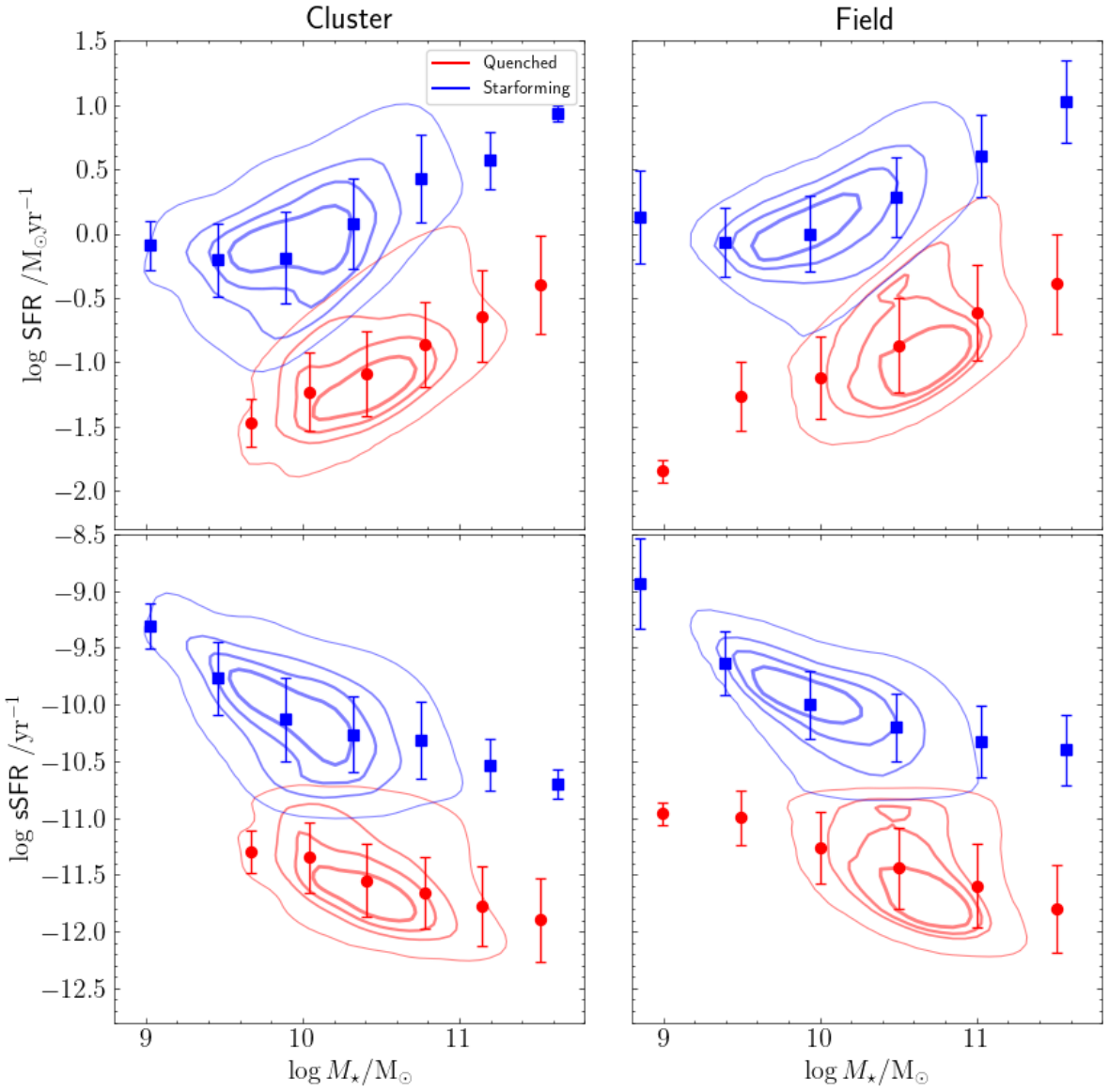
```
ax22.set_ylim(-12.8, -8.5)

ax22.set_xlabel('$\log M_{\star}/\mathrm{M}_\odot$')
ax21.set_xlabel('$\log M_{\star}/\mathrm{M}_\odot$')
ax11.set_ylabel('$\log$ SFR $/\mathrm{M}_{\odot} \mathrm{yr}^{-1}$')
ax21.set_ylabel('$\log$ sSFR $/\mathrm{yr}^{-1}$')

ax11.set_title('Quenched')
ax12.set_title('Starforming')
fig.subplots_adjust(hspace=0.01, wspace=0.1)
```



In [530…

```
fig, axes = plt.subplots(2,2, figsize=(10,10), sharex=True, sharey='row')

for i,n in enumerate([CL, FD]):
    sns.kdeplot(
        ax = axes[0,i],
        data=quenched[n],
        x='stellarmass', y='sfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color='r', alpha=0.5,
    )
    bines_central, median, std = weighted_mean_of_col(data=quenched[n], n
    axes[0,i].errorbar(
        bines_central, median, yerr=std,
        fmt='o',
```

```
        color='r',
    )
    sns.kdeplot(
        ax = axes[0,i],
        data=starforming[n],
        x='stellarmass', y='sfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color='b', alpha=0.5,
    )

    bines_central, median, std = weighted_mean_of_col(data=starforming[n]
    axes[0,i].errorbar(
        bines_central, median, yerr=std,
        fmt='s',
        color='b',
    )
    sns.kdeplot(
        ax = axes[1,i],
        data=quenched[n],
        x='stellarmass', y='ssfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color='r', alpha=0.5
    )
    bines_central, median, std = weighted_mean_of_col(data=quenched[n], n
    axes[1,i].errorbar(
        bines_central, median, yerr=std,
        fmt='o',
        color='r',
    )

    sns.kdeplot(
        ax = axes[1,i],
        data=starforming[n],
        x='stellarmass', y='ssfr', weights='w',
        gridsize=GRID, levels=LEVELS, linewidths=LWS,
        color='b', alpha=0.5
    )
    bines_central, median, std = weighted_mean_of_col(data=starforming[n]
    axes[1,i].errorbar(
        bines_central, median, yerr=std,
        fmt='s',
        color='b',
    )

axes[0,0].plot([],[], lw=1.5, c='r', label='Quenched')
axes[0,0].plot([],[], lw=1.5, c='b', label='Starforming')
axes[0,0].legend()
axes[0,0].set_xlim(8.8, 11.8)
axes[0,0].set_ylim(-2.3, 1.5)
axes[1,1].set_ylim(-12.8, -8.5)

axes[1,1].set_xlabel('$\log M_{\star}/\mathrm{M}_\odot$')
axes[1,0].set_xlabel('$\log M_{\star}/\mathrm{M}_\odot$')
axes[0,0].set_ylabel('$\log$ SFR $/\mathrm{M}_{\odot} \mathrm{yr}^{-1}$')
axes[1,0].set_ylabel('$\log$ sSFR $/\mathrm{yr}^{-1}$')

axes[0,0].set_title('Cluster')
axes[0,1].set_title('Field')
fig.subplots_adjust(hspace=0.01, wspace=0.1)
```



Bines de Masa

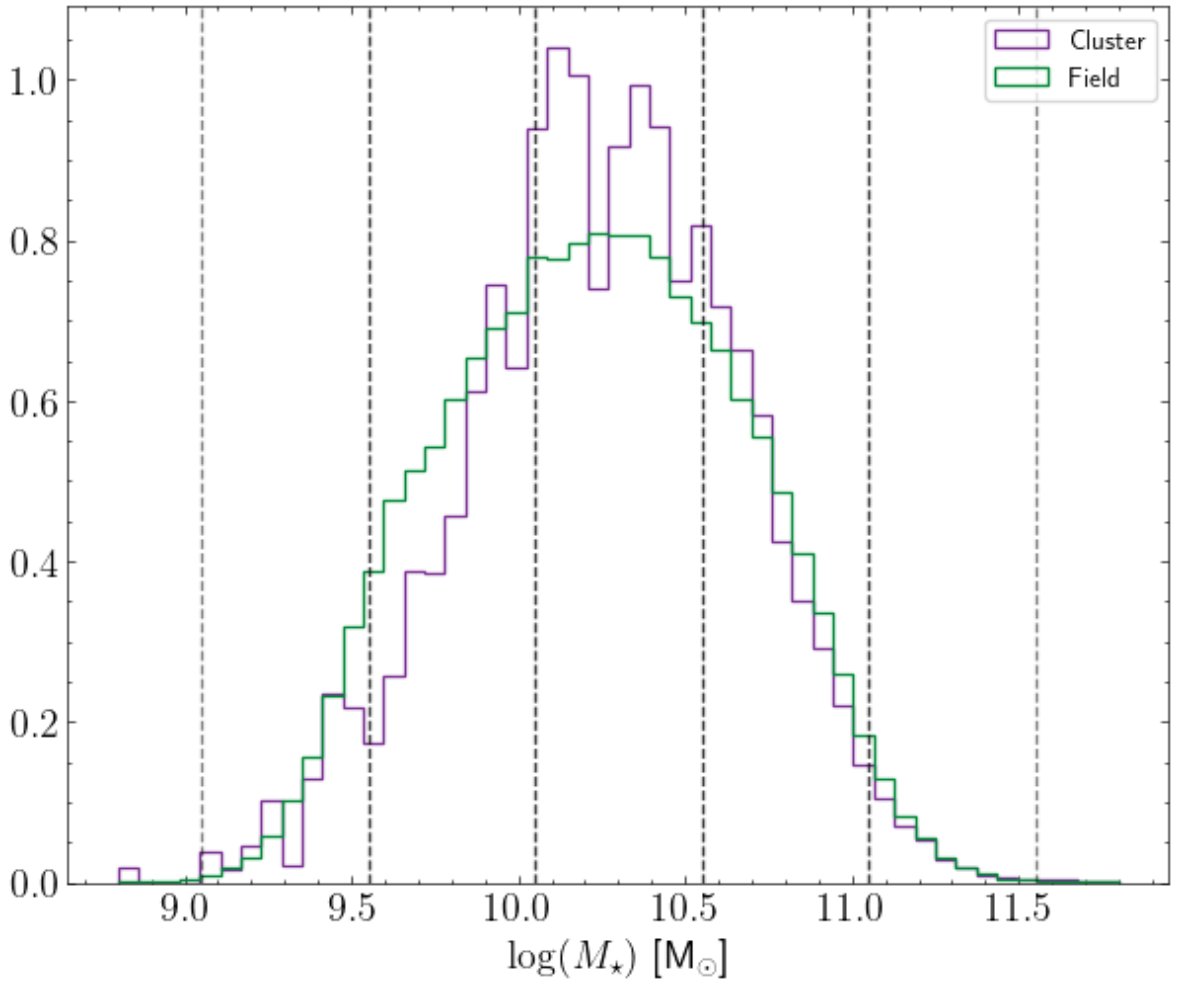
Podemos realizar ahora un analisis en bines de masa estelar...

Se separan en 5 intervalos equiespaciados de $\log M_\star$ entre 8.8 y 11.8.

In [23]:
```
# Separamos en n bines de masa
n_masa = 6
b_mstar = np.linspace(8.8,11.8,n_masa+1)
b_mstar = (b_mstar[:-1] + np.diff(b_mstar)*0.5)
print('Bordes de los bines: ', b_mstar)

mstar = {}
for i in range(n_masa-1):
    massbin = f'{b_mstar[i]:2.2f}-{b_mstar[i+1]:2.2f}'
    mstar[massbin] = {}
    for n in [CL,FD]:
        mstar[massbin][n] = S[n].query(
            f'stellarmass > {b_mstar[i]} and stellarmass < {b_mstar[i+1]}
            inplace=False
        )
```

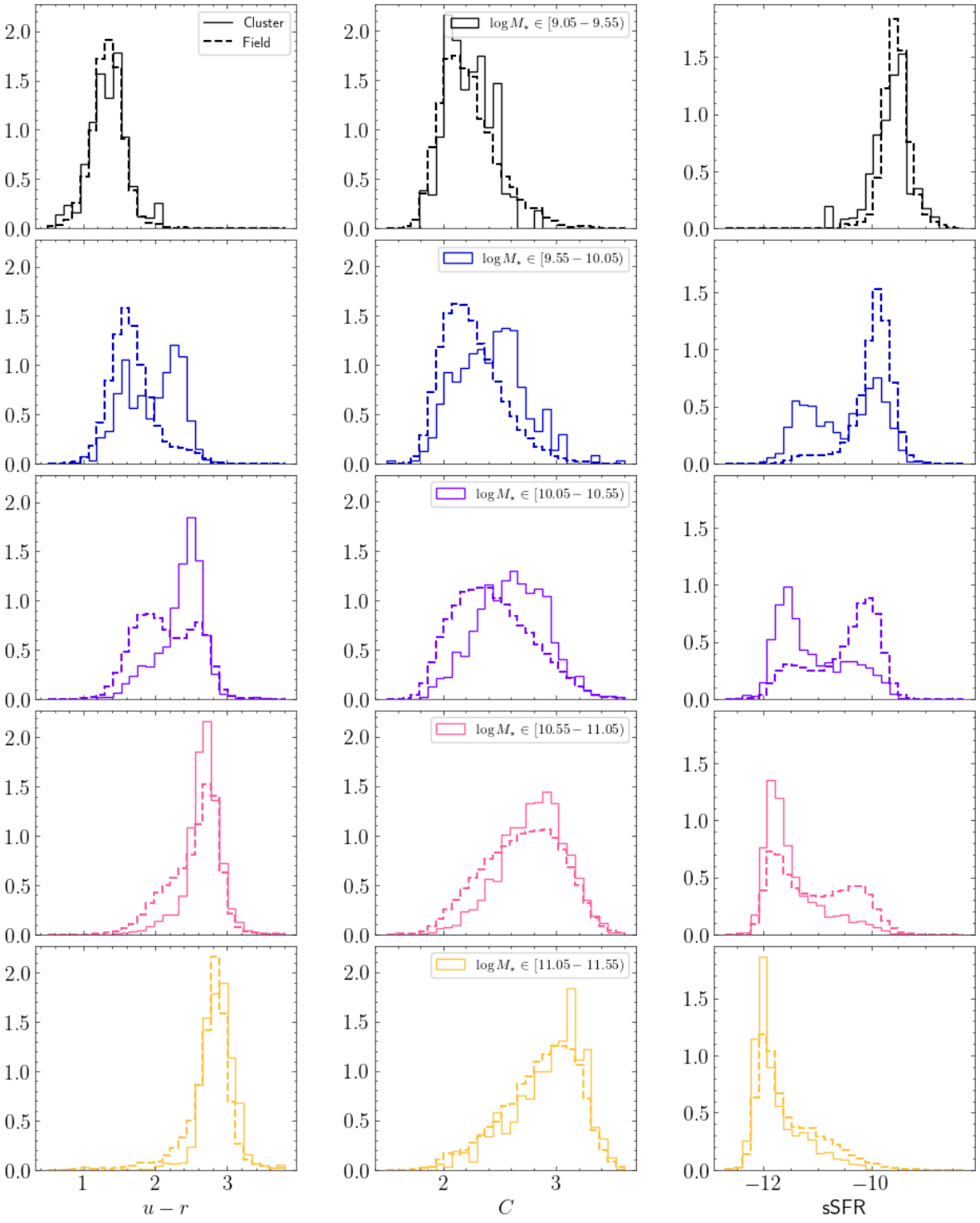
```
Bordes de los bines:  [ 9.05  9.55 10.05 10.55 11.05 11.55]
```

In [24]:
```
bins = np.linspace(8.8, 11.8, 50)
fig, ax = plt.subplots()

for n in [CL,FD]:
    ax.hist(
        S[n]['stellarmass'],
        weights=S[n]['w'],
        bins=bins,
        density=True,
        histtype='step',
        label=n,
        color=COLORS[n]
    )

for b in mstar.keys():
    ax.axvline(float(b.split('-')[0]), ls='--', color='k', alpha=0.5)
    ax.axvline(float(b.split('-')[1]), ls='--', color='k', alpha=0.5)

ax.set_xlabel('$\log(M_{\star})$ [M$_{\odot}$]')
ax.legend();
```



In [25]:
```
# Distribuciones de color de las dif masas
bins_ur = np.linspace(0.5,3.8,30)
bins_c = np.linspace(1.5,3.6,30)
bins_ssfr = np.linspace(-12.7,-8.3,30)
cmap = plt.get_cmap('gnuplot2', n_masa)
fig, axes = plt.subplots(5,3,figsize=(12,15), sharex='col', sharey='col')

axes[0,0].plot([],[],color='k',label=CL)
axes[0,0].plot([],[],ls='--',lw=1.5,color='k',label=FD)
```

```
axes[0,0].legend()
i=0
for b,S_star in mstar.items():
    axes[i,0].hist(S_star[CL]['ur'], bins=bins_ur, weights=S_star[CL]['w'
    axes[i,0].hist(S_star[FD]['ur'], bins=bins_ur, weights=S_star[FD]['w'

    axes[i,1].hist(S_star[CL]['C'], bins=bins_c, weights=S_star[CL]['w'],
    axes[i,1].hist(S_star[FD]['C'], bins=bins_c, weights=S_star[FD]['w'],

    axes[i,2].hist(S_star[CL]['ssfr'], bins=bins_ssfr, weights=S_star[CL]
    axes[i,2].hist(S_star[FD]['ssfr'], bins=bins_ssfr, weights=S_star[FD]

    axes[i,1].legend()
    i+=1

    # Son muestras similares?
    #display(Math(fr'$\log M {{\star}} \in [{b})$'))
    #for col in ['ur', 'C']:
    #    ks_res = ks_2samp(S_star[CL][col], S_star[FD][col], method='exac
    #    print(f'pvalue = {ks_res.pvalue:2.2e}')
    #    if ks_res.pvalue > 0.05:
    #        print(f'{col}: No es posible rechazar la hip. nula\n')
    #    else:
    #        print(f'{col}: Las muestras son diferentes\n')

axes[-1,0].set_xlabel('$u-r$')
axes[-1,1].set_xlabel('$C$')
axes[-1,2].set_xlabel('sSFR')

fig.subplots_adjust(wspace=0.3, hspace=0.05)
```



Es posible observar que para los intervalos de masa en los extremos, las distribuciones de color y concentración son similares, mientras que los tres intervalos centrales presentan las mayores diferencias. Una de ellas es la bimodalidad del color, para $\log M_\star \in [9.55 - 10.05)$ en la muestra de clusters esta bimodalidad está presente y las de campo son en su mayoría azules, para $\log M_\star \in [10.05 - 10.55)$ ahora las galaxias de cluster son en su mayoría rojas mientras que las de campo tienen una fuerte bimodalidad y para $\log M_\star \in [10.55 - 11.05)$ las componentes azules son minoritarias en ambos ambientes, con la población azul de cluster aún menor que la de campo.

Por su parte, se puede observar de la concentración que la población de early-type ($C > 2.5$) se vuelve mayoritaria con la masa para ambos catalogos. En particular para

los intervalos de $\log M_\star \in [9.55 - 10.05)$ y $[10.05 - 10.55)$ se observa una mayor población de late-type en el campo que en los cluster.